

Practical 1 : Creating database employee Create collections

emp_personal_details with emp_id, emp_name, emp_address, emp_DOB, emp_age, emp_mobilenumber.

```
use employee;
switched to db employee
db.createCollection('emp_personal_details');
db.emp_personal_details.insertOne(
{
  emp_id:101,
  emp_name:"Rohit",
  emp_address:"Pune",
  emp_DOB:new Date("2001-05-14"),
  emp_age:24,
  emp_mobilenumber:9855630267});
db.emp_personal_details.find();
```

Output :

```
[ {
  _id: ObjectId('687dec9c0f294ae5b2eec4a9'),
  emp_id: 101,
  emp_name: 'Rohit',
  emp_address: 'Pune',
  emp_DOB: ISODate('2001-05-13T00:00:00.000Z'),
  emp_age: 24,
  emp_mobilenumber: 9475220540 } ]
```

Practical 2 : Create another collection emp_professional_details with emp_id , name, designation, salary, incentive, working hours.

```
db.createCollection('emp_professional_details');  
db.emp_professional_details.insertOne(  
{  
  emp_id: 101,  
  name: "Rohit",  
  designation:"Software Engineer",  
  Salary:500000,  
  incentive:5000,  
  working_hours:9});  
db.emp_professional_details.find();
```

Output :

```
[ {  
  _id: ObjectId('687df3170f294ae5b2eec4aa'),  
  emp_id: 101,  
  name: 'Rohit',  
  designation: 'Software Engineer',  
  Salary: 500000,  
  incentive: 5000,  
  working_hours: 9 } ]
```

Practical 3 :1) Insert 10 records in the collection emp_personal_details and emp_professional_details.

2) Show all the employee having designation manager.

3) Show all these employee having salary 6000.

1) Insert 10 records in the collection emp_personal_details and emp_professional_details

```
db.emp_personal_details.insertMany( [
  {emp_id:101,emp_name:"Sunil",Designation:"Manager",Salary:40000},
  {emp_id:102,emp_name:"Rita",Designation:"Cloud Computing",Salary:6000},
  {emp_id:103,emp_name:"Rahul",Designation:"Manager",Salary:5000},
  {emp_id:104,emp_name:"Neha",Designation:"Web Developer",Salary:6000},
  {emp_id:105,emp_name:"Hitesh",Designation:"Full Stack Developer",Salary:5000},

  {emp_id:106,emp_name:"Neha",Designation:"Web Developer",Salary:40000},
  {emp_id:107,emp_name:"Sheetal",Designation:"Cloud Computing",Salary:5000},
  {emp_id:108,emp_name:"Tina",Designation:"IT",Salary:4000},
  {emp_id:109,emp_name:"Girish",Designation:"Web Developer",Salary:7000},
  {emp_id:110,emp_name:"Nikhil",Designation:"Full Stack Developer",Salary:5000}]);
```

Output:

```
[ {
  _id: ObjectId('687e03abf6d517abbbeec4ae'),
  emp_id: 101,
  emp_name: 'Sunil',
  Designation: 'Manager',
  Salary: 40000
},
{
  _id: ObjectId('687e03abf6d517abbbeec4af'),
  emp_id: 102,
  emp_name: 'Rita',
  Designation: 'Cloud Computer',
```

```
Salary: 6000
},
{
  _id: ObjectId('687e03abf6d517abbbeec4b0'),
  emp_id: 103,
  emp_name: 'Rahul',
  Designation: 'Manager',
  Salary: 5000
},
{
  _id: ObjectId('687e03abf6d517abbbeec4b1'),
  emp_id: 104,
  emp_name: 'Neha',
  Designation: 'Web Developer',
  Salary: 6000
},
{
  _id: ObjectId('687e03abf6d517abbbeec4b2'),
  emp_id: 105,
  emp_name: 'Hitesh',
  Designation: 'Full Stack Developer',
  Salary: 5000
}
{
  _id: ObjectId('687e03abf6d517abbbeec4b1'),
  emp_id: 106,
  emp_name: 'Neha',
  Designation: 'Web Developer',
  Salary: 40000
}
```

```
},  
{  
  _id: ObjectId('687e03abf6d517abbbbec4b1'),  
  emp_id: 107,  
  emp_name: 'Sheetal',  
  Designation: 'Cloud Computing',  
  Salary: 5000  
},  
{  
  _id: ObjectId('687e03abf6d517abbbbec4b1'),  
  emp_id: 108,  
  emp_name: 'Tina',  
  Designation: 'IT',  
  Salary: 4000  
},  
{  
  _id: ObjectId('687e03abf6d517abbbbec4b1'),  
  emp_id: 109,  
  emp_name: 'Girish',  
  Designation: 'Web Developer',  
  Salary: 7000  
},  
{  
  _id: ObjectId('687e03abf6d517abbbbec4b1'),  
  emp_id: 110,  
  emp_name: 'Nikhil',  
  Designation: 'Full Stack Developer',  
  Salary: 5000  
}
```

```
]
```

2) Show all the employee having designation manager.

```
db.emp_professional_details.find({Designation:"Manager"});
```

Output :

```
[
  {
    _id: ObjectId('687e06ddf6d517abbbeec4b3'),
    emp_id: 101,
    emp_name: 'Sunil',
    Designation: 'Manager',
    Salary: 40000
  },
  {
    _id: ObjectId('687e06ddf6d517abbbeec4b5'),
    emp_id: 103,
    emp_name: 'Rahul',
    Designation: 'Manager',
    Salary: 5000
  }
]
```

3) Show all the employee having salary 6000.

```
db.emp_professional_details.find({Salary:6000});
```

Output :

```
[
  {
    _id: ObjectId('687e06ddf6d517abbbeec4b4'),
    emp_id: 102,
    emp_name: 'Rita',
    Designation: 'Cloud Computer',
  }
]
```

```
Salary: 6000
},
{
  _id: ObjectId('687e06ddf6d517abbbeec4b6'),
  emp_id: 104,
  emp_name: 'Neha',
  Designation: 'Web Developer',
  Salary: 6000
}
]
```

Practical 4 : Update the collection emp_personal_details,add field status and set it to retired where age is greater than 60.

1) Update collection emp_professional_details , give incentive 5000 to employee whose working hours is greater than 45 per week.

2) Add 1000 to salary employee whose designation is accountant.

```
db.emp_personal_details.insertMany( [  
  {emp_id:101,emp_name:"Sunil",Designation:"Manager",Salary:40000,age:61,working_h  
    ours:46},
```

```
  {emp_id:102,emp_name:"Rita",Designation:"Cloud  
    Computer",Salary:6000,age:50,working_hours:40},
```

```
  {emp_id:103,emp_name:"Rahul",Designation:"accountant",Salary:5000,age:66,working_  
    hours:44},
```

```
  {emp_id:104,emp_name:"Neha",Designation:"Web  
    Developer",Salary:6000,age:40,working_hours:30},
```

```
  {emp_id:105,emp_name:"Hitesh",Designation:"accountant",Salary:5000,age:64,working_  
    _hours:47} ]]);
```

```
//Update emp_personal_details: add field "status" = "retired" where emp_age > 60
```

```
db.emp_personal_details.updateMany(  
  {$age: {$gt:60}},
```

```
  {$set: {status:"retired"}}
```

```
);
```

```
);
```

```
db.emp_personal_details.find();
```

Output :

```
{  
  _id: ObjectId('687e0ff9f6d517abbbbec4bd'),  
  emp_id: 101,  
  emp_name: 'Sunil',  
  Designation: 'Manager',  
  Salary: 40000,  
  age: 61,  
  incentive: 5000,  
  working_hours: 46,
```



```

    status: 'retired'
  },
  {
    _id: ObjectId('687e0ff9f6d517abbbeec4bf'),
    emp_id: 103,
    emp_name: 'Rahul',
    Designation: 'accountant',
    Salary: 5000,
    age: 66,
    incentive: 3000,
    working_hours: 44,
    status: 'retired'
  },
  {
    _id: ObjectId('687e0cacf6d517abbbeec4bc'),
    emp_id: 105,
    emp_name: 'Hitesh',
    Designation: 'Full Stack Developer',
    Salary: 5000,
    age: 64,
    status: 'retired'
  }
]

```

1) Update collection emp_professional_details , give incentive 5000 to employee whose working hours is greater than 45 per week.

```

db.emp_professional_details.updateMany(
  { working_hours: { $gt: 45 } },
  { $set: { incentive: 5000 } }
)

```

```
);  
db.emp_professional_details.find();
```

Output:

```
[  
  {  
    _id: ObjectId('687e1495f6d517abbbeec4cc'),  
    emp_id: 101,  
    emp_name: 'Sunil',  
    Designation: 'Manager',  
    Salary: 40000,  
    age: 61,  
    working_hours: 46,  
    incentive: 5000  
  },  
  {  
    _id: ObjectId('687e1417f6d517abbbeec4cb'),  
    emp_id: 105,  
    emp_name: 'Hitesh',  
    Designation: 'accountant',  
    Salary: 5000,  
    age: 64,  
    incentive: 5000,  
    working_hours: 47  
  }  
]
```

2) Add 1000 to salary employee whose designation is accountant.

```
employee> db.emp_professional_details.updateMany(  
  { Designation:"accountant" } ,  
  {$inc:{Salary:1000} }  
);
```

```
employee> db.emp_professional_details.find();
```

Output:

```
{  
  _id: ObjectId('687e1495f6d517abbbeec4ce'),  
  emp_id: 103,
```

```
emp_name: 'Rahul',
Designation: 'accountant',
Salary: 6000,
age: 66,
working_hours: 44
},
{
  _id: ObjectId('687e1417f6d517abbbeec4cb'),
  emp_id: 105,
  emp_name: 'Hitesh',
  Designation: 'accountant',
  Salary: 6000,
  age: 64,
  incentive: 5000,
  working_hours: 47
}]
```

Practical 5 : Create index on emp_id in collection emp_professional_details .

2. Create multiple index on emp_id,emp_name in collection emp_professional_details

```
db.emp.insertMany( [{ emp_id:1,name: "Sunil", designation: "clerk", salary: 4000,mob:685866969 },
{ emp_id:2,name: "Neha", designation: "Software Engineer", salary: 5000,mob:4758485949 },
{ emp_id:3,name: "Sumita", designation: "clerk", salary: 8000,mob:9796858796 },
{ emp_id:4,name: "Rakesh",designation: "Data Analyst", salary: 5000,mob:6859605060 },
{ emp_id:5,name: "Subhash", designation: "Software Engineer", salary: 6000,mob:8667887876 }]);
```

2) Create multiple indexes on emp_id,emp_name in collection emp_professional_details.

```
db.emp.createIndex( { emp_id: 1,name:1})
```

```
db.emp.getIndexes();
```

```
db.emp.find();
```

Output :

```
[
  { v: 2, key: { _id: 1 }, name: '_id' },
  { v: 2, key: { emp_id: 1 }, name: 'emp_id_1' },
  { v: 2, key: { emp_id: 1, name: 1 }, name: 'emp_id_1_name_1' }
]
{
  _id: ObjectId('688db9842828f2dba1eec4b0'),
  name: 'Sumita',
  designation: 'clerk',
  salary: 8000,
  mob: 9796858796
},
{
  _id: ObjectId('688db9842828f2dba1eec4b1'),
  name: 'Rakesh',
```

```
    designation: 'Data Analyst',
    salary: 5000,
    mob: 6859605060
  },
  {
    _id: ObjectId('688db9842828f2dba1eec4b2'),
    name: 'Subhash',
    designation: 'Software Engineer',
    salary: 6000,
    mob: 57485886858
  },
  {
    _id: ObjectId('688dba012828f2dba1eec4b3'),
    name: 'Sunil',
    designation: 'clerk',
    salary: 4000,
    mob: 685866969
  },
  {
    _id: ObjectId('688dba012828f2dba1eec4b4'),
    name: 'Neha',
    designation: 'Software Engineer',
    salary: 5000,
    mob: 4758485949
  },
  {
    _id: ObjectId('688dba012828f2dba1eec4b5'),
    name: 'Sumita',
    designation: 'clerk',
    salary: 8000,
    mob: 9796858796
  }
}
```

Practical 6 :

1. Find sum of salaries of employees having designation clerk.

2. Filter the employees having the designation software engineer and find the minimum salary.

```
db.emp.insertMany( [  
  { name: "Sunil", designation: "clerk", salary: 4000,mob:685866969 },  
  { name: "Neha", designation: "Software Engineer", salary: 5000,mob:4758485949 },  
  { name: "Sumita", designation: "clerk", salary: 8000,mob:9796858796 },  
  { name: "Rakesh",designation: "Data Analyst", salary: 5000,mob:6859605060 }, { name:  
"Subhash", designation: "Software Engineer", salary: 6000,mob:57485886858 }  
]);
```

1) Find sum of salaries of employee having designation clerk.

```
db.emp.aggregate([  
  { $match: { designation: "clerk" } },  
  { $group: { _id: null, totalsalary: { $sum: "$salary" } } }  
]);
```

Output :

```
[ { _id: null, totalsalary: 12000 } ]
```

2) Filter the employee having designation software engineer and find the minimum salary.

```
db.emp.aggregate([  
  { $match: { designation: "Software Engineer" } },  
  { $group: { _id: null, minsalary: { $min: "$salary" } }  
} ]]);
```

Output :

```
[ { _id: null, minsalary: 5000 } ]
```

Practical 7: Use unwind command and show the employees whose mobile number is stored in array

2. Use skip command to skip first 3 records and display rest of records .

3. Use limit command to show only first four records of collection

```
db.emp.insertMany( [
  { emp_id:1,name: "Sunil", designation: "clerk", salary: 4000,mob:685866969 }, {
  emp_id:2,name: "Neha", designation: "Software Engineer", salary: 5000,mob:4758485949 },
  { emp_id:3,name: "Sumita", designation: "clerk", salary: 8000,mob:9796858796 },
  { emp_id:4,name: "Rakesh",designation: "Data Analyst", salary: 5000,mob:6859605060 },
  { emp_id:5,name: "Subhash", designation: "Software Engineer", salary:
  6000,mob:["57485886858","8667887876"]}
];
```

1) Use unwind command and show the employee whose mobile number is stored in array.

```
db.emp.aggregate([
  {$unwind:"$mob"}
]);
```

Output :

```
{
  _id: ObjectId('688dbfbc702d53b13aee4ad'),
  emp_id: 5,
  name: 'Subhash',
  designation: 'Software Engineer',
  salary: 6000,
  mob: '57485886858'
},
{
  _id: ObjectId('688dbfbc702d53b13aee4ad'),
```

```
emp_id: 5,  
name: 'Subhash',  
designation: 'Software Engineer',  
salary: 6000,  
mob: '8667887876'  
}
```

2) Use skip command to skip first 3 records and display rest of records.

```
db.emp.aggregate([  
  {$skip:3}  
]);
```

Output :

```
[  
  {  
    _id: ObjectId('688dbfbc702d53b13aeec4ac'),  
    emp_id: 4,  
    name: 'Rakesh',  
    designation: 'Data Analyst',  
    salary: 5000,  
    mob: 6859605060  
  },  
  {  
    _id: ObjectId('688dbfbc702d53b13aeec4ad'),  
    emp_id: 5,  
    name: 'Subhash',  
    designation: 'Software Engineer',  
    salary: 6000,  
    mob: [ '57485886858', '8667887876' ]  
  }  
]
```


3) Use limit command to show only first 4 records of collection.

```
db.emp.aggregate([  
  {$limit:4}  
]);
```

Output :

```
[  
  {  
    _id: ObjectId('688dbfbc702d53b13aeec4a9'),  
    emp_id: 1,  
    name: 'Sunil',  
    designation: 'clerk',  
    salary: 4000,  
    mob: 685866969  
  },  
  {  
    _id: ObjectId('688dbfbc702d53b13aeec4aa'),  
    emp_id: 2,  
    name: 'Neha',  
    designation: 'Software Engineer',  
    salary: 5000,  
    mob: 4758485949  
  },  
  {  
    _id: ObjectId('688dbfbc702d53b13aeec4ab'),  
    emp_id: 3,  
    name: 'Sumita',  
    designation: 'clerk',  
    salary: 8000,  
    mob: 9796858796  
  }  
]
```

```
},  
{  
  _id: ObjectId('688dbfbc702d53b13aeec4ac'),  
  emp_id: 4,  
  name: 'Rakesh',  
  designation: 'Data Analyst',  
  salary: 5000,  
  mob: 6859605060  
}  
]
```

Practical 8 : Create replica set of employee database and insert records in primary node and display the same records in secondary nodes.

Step 1: Start MongoDB with replica set enabled (single node)

```
mongod --port 27017 --dbpath /data/db1 --replSet rs0 --bind_ip localhost
```

// Step 2: Connect to mongo shell on primary

```
mongo --port 27017
```

// Step 3: Initialize the replica set

```
rs.initiate({  
  _id: "rs0",  
  members: [{ _id: 0, host: "localhost:27017" }]  
})
```

```
rs.status()
```

// Step 4: Use employee database

```
use employee
```

// Insert sample records into emp_personal_details collection

```
db.emp_personal_details.insertMany([  
  { emp_id: 1, emp_name: "Amit Sharma", emp_address: "Delhi" },  
  { emp_id: 2, emp_name: "Priya Singh", emp_address: "Mumbai" }  
])
```

```
// Step 5: Connect to the same node (acts as secondary here)
```

```
// (In single node setup, primary and secondary are same)
```

```
// Set read preference to secondary (for learning purpose)
```

```
db.getMongo().setReadPref('secondary')
```

```
// Read the inserted records from emp_personal_details
```

```
db.emp_personal_details.find()
```

Output :

```
rs0 [direct: primary] test> use employee
```

```
switched to db employee
```

```
rs0 [direct: primary] employee> db.emp_personal_details.find()
```

```
[
```

```
  { _id: ObjectId("652c2f1d8f9d3b1c2e34a001"), emp_id: 1, emp_name: "Amit Sharma",  
    emp_address: "Delhi" },
```

```
  { _id: ObjectId("652c2f1d8f9d3b1c2e34a002"), emp_id: 2, emp_name: "Priya Singh",  
    emp_address: "Mumbai" }
```

```
]
```

Practical 9 : Create a MongoDB collection named restaurants to store the following information about restaurants:

Building number

Street name

Zip code Coordinates (longitude and latitude)

Borough

Cuisine type

Grades (each grade includes: date, grade (A/B/C), and score)

// Use your database (e.g., restaurantDB)

use restaurantDB

// Insert sample data into restaurants collection

```
db.restaurants.insertMany([
  {
    building_number: "123",
    street_name: "MG Road",
    zip_code: "400001",
    coordinates: { longitude: 73.8567, latitude: 18.5204 },
    borough: "Mumbai",
    cuisine_type: "Indian",
    grades: [
      { date: ISODate("2024-01-15"), grade: "A", score: 95 },
      { date: ISODate("2023-06-10"), grade: "B", score: 88 }
    ]
  },
  {
    building_number: "45",
    street_name: "Brigade Road",
```

```
zip_code: "560001",
coordinates: { longitude: 77.5946, latitude: 12.9716 },
borough: "Bangalore",
cuisine_type: "Continental",
grades: [
  { date: ISODate("2024-02-20"), grade: "A", score: 92 },
  { date: ISODate("2023-08-18"), grade: "A", score: 90 }
],
},
{
  building_number: "67",
  street_name: "Connaught Place",
  zip_code: "110001",
  coordinates: { longitude: 77.2167, latitude: 28.6667 },
  borough: "Delhi",
  cuisine_type: "Chinese",
  grades: [
    { date: ISODate("2023-12-05"), grade: "B", score: 85 },
    { date: ISODate("2023-05-22"), grade: "C", score: 75 }
  ],
},
{
  building: "1007",
  street: "Morris Park Ave",
  zipcode: "10462",
  coordinates: [-73.856077, 40.848447],
  borough: "Bronx",
  cuisine: "Bakery",
  grades: [
```

```
{ date: new Date("2014-03-03"), grade: "A", score: 2 },
{ date: new Date("2013-09-11"), grade: "A", score: 6 }
]
},
{
  building: "469",
  street: "Flatbush Avenue",
  zipcode: "11225",
  coordinates: [-73.961704, 40.662942],
  borough: "Brooklyn",
  cuisine: "Hamburgers",
  grades: [
    { date: new Date("2014-12-30"), grade: "A", score: 8 },
    { date: new Date("2014-07-01"), grade: "B", score: 23 }
  ]
},
{
  building: "97-22",
  street: "63rd Road",
  zipcode: "11374",
  coordinates: [-73.860115, 40.731173],
  borough: "Queens",
  cuisine: "American",
  grades: [
    { date: new Date("2014-11-15"), grade: "A", score: 10 },
    { date: new Date("2013-05-23"), grade: "B", score: 17 }
  ]
},
{
```

```
building: "8825",
street: "Astoria Boulevard",
zipcode: "11369",
coordinates: [-73.878611, 40.764425],
borough: "Queens",
cuisine: "Italian",
grades: [
  { date: new Date("2014-06-18"), grade: "A", score: 9 },
  { date: new Date("2013-12-20"), grade: "B", score: 18 }
],
},
{
  building: "2206",
  street: "Victory Boulevard",
  zipcode: "10314",
  coordinates: [-74.137728, 40.611957],
  borough: "Staten Island",
  cuisine: "Chinese",
  grades: [
    { date: new Date("2014-04-10"), grade: "A", score: 7 },
    { date: new Date("2013-09-15"), grade: "B", score: 16 }
  ],
},
{
  building: "188",
  street: "Bay Street",
  zipcode: "10301",
  coordinates: [-74.076132, 40.637642],
  borough: "Staten Island",
```



```

    cuisine: "Seafood",
    grades: [
      { date: new Date("2014-08-22"), grade: "A", score: 5 },
      { date: new Date("2013-10-11"), grade: "A", score: 12 }
    ]
  },
  {
    building: "15",
    street: "West 46th Street",
    zipcode: "10036",
    coordinates: [-73.981232, 40.757784],
    borough: "Manhattan",
    cuisine: "Japanese",
    grades: [
      { date: new Date("2014-11-03"), grade: "A", score: 3 },
      { date: new Date("2013-05-19"), grade: "B", score: 14 }
    ]
  }
]
)

```

// Find and display all restaurants

```
db.restaurants.find()
```

Output :

```

[
  {
    _id: ObjectId('68edebda344eaff523eec4a9'),
    building_number: '123',
    street_name: 'MG Road',
    zip_code: '400001',

```

```
coordinates: { longitude: 73.8567, latitude: 18.5204 },
borough: 'Mumbai',
cuisine_type: 'Indian',
grades: [
  {
    date: ISODate('2024-01-15T00:00:00.000Z'),
    grade: 'A',
    score: 95
  },
  {
    date: ISODate('2023-06-10T00:00:00.000Z'),
    grade: 'B',
    score: 88
  }
],
},
{
  _id: ObjectId('68edebda344eaff523eec4aa'),
  building_number: '45',
  street_name: 'Brigade Road',
  zip_code: '560001',
  coordinates: { longitude: 77.5946, latitude: 12.9716 },
  borough: 'Bangalore',
  cuisine_type: 'Continental',
  grades: [
    {
      date: ISODate('2024-02-20T00:00:00.000Z'),
      grade: 'A',
      score: 92
```

```
    },
    {
      date: ISODate('2023-08-18T00:00:00.000Z'),
      grade: 'A',
      score: 90
    }
  ]
},
{
  _id: ObjectId('68edebda344eaff523eec4ab'),
  building_number: '67',
  street_name: 'Connaught Place',
  zip_code: '110001',
  coordinates: { longitude: 77.2167, latitude: 28.6667 },
  borough: 'Delhi',
  cuisine_type: 'Chinese',
  grades: [
    {
      date: ISODate('2023-12-05T00:00:00.000Z'),
      grade: 'B',
      score: 85
    },
    {
      date: ISODate('2023-05-22T00:00:00.000Z'),
      grade: 'C',
      score: 75
    }
  ]
},
```

```
{
  _id: ObjectId('68edebda344eaff523eec4ac'),
  building: '1007',
  street: 'Morris Park Ave',
  zipcode: '10462',
  coordinates: [ -73.856077, 40.848447 ],
  borough: 'Bronx',
  cuisine: 'Bakery',
  grades: [
    {
      date: ISODate('2014-03-03T00:00:00.000Z'),
      grade: 'A',
      score: 2
    },
    {
      date: ISODate('2013-09-11T00:00:00.000Z'),
      grade: 'A',
      score: 6
    }
  ]
},
{
  _id: ObjectId('68edebda344eaff523eec4ad'),
  building: '469',
  street: 'Flatbush Avenue',
  zipcode: '11225',
  coordinates: [ -73.961704, 40.662942 ],
  borough: 'Brooklyn',
  cuisine: 'Hamburgers',
```

```
grades: [  
  {  
    date: ISODate('2014-12-30T00:00:00.000Z'),  
    grade: 'A',  
    score: 8  
  },  
  {  
    date: ISODate('2014-07-01T00:00:00.000Z'),  
    grade: 'B',  
    score: 23  
  }  
]  
,  
{  
  _id: ObjectId('68edebda344eaff523eec4ae'),  
  building: '97-22',  
  street: '63rd Road',  
  zipcode: '11374',  
  coordinates: [ -73.860115, 40.731173 ],  
  borough: 'Queens',  
  cuisine: 'American',  
  grades: [  
    {  
      date: ISODate('2014-11-15T00:00:00.000Z'),  
      grade: 'A',  
      score: 10  
    },  
    {  
      date: ISODate('2013-05-23T00:00:00.000Z'),
```

```
    grade: 'B',
    score: 17
  }
]
},
{
  _id: ObjectId('68edebda344eaff523eec4af'),
  building: '8825',
  street: 'Astoria Boulevard',
  zipcode: '11369',
  coordinates: [ -73.878611, 40.764425 ],
  borough: 'Queens',
  cuisine: 'Italian',
  grades: [
    {
      date: ISODate('2014-06-18T00:00:00.000Z'),
      grade: 'A',
      score: 9
    },
    {
      date: ISODate('2013-12-20T00:00:00.000Z'),
      grade: 'B',
      score: 18
    }
  ]
},
{
  _id: ObjectId('68edebda344eaff523eec4b0'),
  building: '2206',
```

```
street: 'Victory Boulevard',
zipcode: '10314',
coordinates: [ -74.137728, 40.611957 ],
borough: 'Staten Island',
cuisine: 'Chinese',
grades: [
  {
    date: ISODate('2014-04-10T00:00:00.000Z'),
    grade: 'A',
    score: 7
  },
  {
    date: ISODate('2013-09-15T00:00:00.000Z'),
    grade: 'B',
    score: 16
  }
],
},
{
  _id: ObjectId('68edebda344eaff523eec4b1'),
  building: '188',
  street: 'Bay Street',
  zipcode: '10301',
  coordinates: [ -74.076132, 40.637642 ],
  borough: 'Staten Island',
  cuisine: 'Seafood',
  grades: [
    {
      date: ISODate('2014-08-22T00:00:00.000Z'),
```

```
    grade: 'A',
    score: 5
  },
  {
    date: ISODate('2013-10-11T00:00:00.000Z'),
    grade: 'A',
    score: 12
  }
]
},
{
  _id: ObjectId('68edebda344eaff523eec4b2'),
  building: '15',
  street: 'West 46th Street',
  zipcode: '10036',
  coordinates: [ -73.981232, 40.757784 ],
  borough: 'Manhattan',
  cuisine: 'Japanese',
  grades: [
    {
      date: ISODate('2014-11-03T00:00:00.000Z'),
      grade: 'A',
      score: 3
    },
    {
      date: ISODate('2013-05-19T00:00:00.000Z'),
      grade: 'B',
      score: 14
    }
  ]
}
```


Practical 10 : 1. Write a MongoDB query to display all the documents in the collection restaurants

2. Write a MongoDB query to display the fields, restaurant_id, name, borough and cuisine for all the documents in the collection restaurant.

```
// Use database
```

```
use restaurantDB
```

```
// Insert sample data into restaurants collection
```

```
db.restaurants.insertMany([
```

```
{
```

```
  restaurant_id: "R1001",
```

```
  name: "Golden Dragon",
```

```
  building: "97",
```

```
  street: "Bowery",
```

```
  zipcode: "10002",
```

```
  borough: "Manhattan",
```

```
  cuisine: "Chinese",
```

```
  coordinates: [-73.9925, 40.7163],
```

```
  grades: [
```

```
    { date: ISODate("2024-01-12"), grade: "A", score: 10 },
```

```
    { date: ISODate("2023-07-10"), grade: "B", score: 17 }  
  ]
```

```
},
```

```
{
```

```
  restaurant_id: "R1002",
```

```
  name: "Spice Heaven",
```

```
  building: "43",
```

```
  street: "Main Street",
```

```
  zipcode: "11375",
```

```
    borough: "Queens",
    cuisine: "Indian",
    coordinates: [-73.8448, 40.7098],
    grades: [
      { date: ISODate("2024-02-15"), grade: "A", score: 8 },
      { date: ISODate("2023-08-11"), grade: "A", score: 12 }
    ]
  },
  {
    restaurant_id: "R1003",
    name: "Bella Roma",
    building: "12",
    street: "Court Street",
    zipcode: "11201",
    borough: "Brooklyn",
    cuisine: "Italian",
    coordinates: [-73.989, 40.692],
    grades: [
      { date: ISODate("2024-03-10"), grade: "B", score: 15 },
      { date: ISODate("2023-09-18"), grade: "A", score: 9 }
    ]
  },
  {
    restaurant_id: "R1004",
    name: "Taco Fiesta",
    building: "25",
    street: "Third Avenue",
    zipcode: "10451",
    borough: "Bronx",
```

```
cuisine: "Mexican",
coordinates: [-73.919, 40.827],
grades: [
  { date: ISODate("2024-04-05"), grade: "A", score: 7 },
  { date: ISODate("2023-10-20"), grade: "B", score: 14 }
],
},
```

```
{
  restaurant_id: "R1005",
  name: "Sushi Zen",
  building: "221",
  street: "Park Avenue",
  zipcode: "10016",
  borough: "Manhattan",
  cuisine: "Japanese",
  coordinates: [-73.981, 40.749],
  grades: [
    { date: ISODate("2024-05-02"), grade: "A", score: 5 },
    { date: ISODate("2023-11-15"), grade: "A", score: 8 }
  ],
},
```

```
{
  restaurant_id: "R1006",
  name: "Green Garden",
  building: "58",
  street: "Queens Boulevard",
  zipcode: "11373",
  borough: "Queens",
  cuisine: "Vegetarian",
```

```
coordinates: [-73.876, 40.737],
grades: [
  { date: ISODate("2024-06-18"), grade: "A", score: 9 },
  { date: ISODate("2023-12-08"), grade: "B", score: 16 }
],
{
  restaurant_id: "R1007",
  name: "Burger Hub",
  building: "9",
  street: "Richmond Avenue",
  zipcode: "10312",
  borough: "Staten Island",
  cuisine: "American",
  coordinates: [-74.176, 40.543],
  grades: [
    { date: ISODate("2024-07-12"), grade: "A", score: 6 },
    { date: ISODate("2024-01-03"), grade: "A", score: 9 }
  ],
{
  restaurant_id: "R1008",
  name: "Paris Café",
  building: "105",
  street: "Broadway",
  zipcode: "10006",
  borough: "Manhattan",
  cuisine: "French",
  coordinates: [-74.013, 40.707],
```

```
grades: [
  { date: ISODate("2024-08-22"), grade: "A", score: 4 },
  { date: ISODate("2024-02-10"), grade: "A", score: 7 }
],
{
  restaurant_id: "R1009",
  name: "Royal Biryani",
  building: "17",
  street: "Atlantic Avenue",
  zipcode: "11217",
  borough: "Brooklyn",
  cuisine: "Indian",
  coordinates: [-73.977, 40.683],
  grades: [
    { date: ISODate("2024-09-11"), grade: "B", score: 13 },
    { date: ISODate("2024-03-07"), grade: "A", score: 9 }
  ],
  {
    restaurant_id: "R1010",
    name: "Ocean Grill",
    building: "65",
    street: "Grand Concourse",
    zipcode: "10452",
    borough: "Bronx",
    cuisine: "Seafood",
    coordinates: [-73.921, 40.838],
    grades: [
```

```
    { date: ISODate("2024-10-01"), grade: "A", score: 8 },
    { date: ISODate("2024-04-15"), grade: "B", score: 15 }
  ]
}
]);

// 1. Display all documents in the restaurants collection
db.restaurants.find()
```

Output :

```
[
  {
    _id: ObjectId('68edee02344eaff523eec4b3'),
    restaurant_id: 'R1001',
    name: 'Golden Dragon',
    building: '97',
    street: 'Bowery',
    zipcode: '10002',
    borough: 'Manhattan',
    cuisine: 'Chinese',
    coordinates: [ -73.9925, 40.7163 ],
    grades: [
      {
        date: ISODate('2024-01-12T00:00:00.000Z'),
        grade: 'A',
        score: 10
      },
      {
        date: ISODate('2023-07-10T00:00:00.000Z'),
```

```
    grade: 'B',
    score: 17
  }
]
},
{
  _id: ObjectId('68edee02344eaff523eec4b4'),
  restaurant_id: 'R1002',
  name: 'Spice Heaven',
  building: '43',
  street: 'Main Street',
  zipcode: '11375',
  borough: 'Queens',
  cuisine: 'Indian',
  coordinates: [ -73.8448, 40.7098 ],
  grades: [
    {
      date: ISODate('2024-02-15T00:00:00.000Z'),
      grade: 'A',
      score: 8
    },
    {
      date: ISODate('2023-08-11T00:00:00.000Z'),
      grade: 'A',
      score: 12
    }
  ]
},
{
```

```
_id: ObjectId('68edee02344eaff523eec4b5'),
restaurant_id: 'R1003',
name: 'Bella Roma',
building: '12',
street: 'Court Street',
zipcode: '11201',
borough: 'Brooklyn',
cuisine: 'Italian',
coordinates: [ -73.989, 40.692 ],
grades: [
  {
    date: ISODate('2024-03-10T00:00:00.000Z'),
    grade: 'B',
    score: 15
  },
  {
    date: ISODate('2023-09-18T00:00:00.000Z'),
    grade: 'A',
    score: 9
  }
],
},
{
  _id: ObjectId('68edee02344eaff523eec4b6'),
  restaurant_id: 'R1004',
  name: 'Taco Fiesta',
  building: '25',
  street: 'Third Avenue',
  zipcode: '10451',
```



```
borough: 'Bronx',
cuisine: 'Mexican',
coordinates: [ -73.919, 40.827 ],
grades: [
  {
    date: ISODate('2024-04-05T00:00:00.000Z'),
    grade: 'A',
    score: 7
  },
  {
    date: ISODate('2023-10-20T00:00:00.000Z'),
    grade: 'B',
    score: 14
  }
],
},
{
  _id: ObjectId('68edee02344eaff523eec4b7'),
  restaurant_id: 'R1005',
  name: 'Sushi Zen',
  building: '221',
  street: 'Park Avenue',
  zipcode: '10016',
  borough: 'Manhattan',
  cuisine: 'Japanese',
  coordinates: [ -73.981, 40.749 ],
  grades: [
    {
      date: ISODate('2024-05-02T00:00:00.000Z'),
```

```
    grade: 'A',
    score: 5
  },
  {
    date: ISODate('2023-11-15T00:00:00.000Z'),
    grade: 'A',
    score: 8
  }
]
},
{
  _id: ObjectId('68edee02344eaff523eec4b8'),
  restaurant_id: 'R1006',
  name: 'Green Garden',
  building: '58',
  street: 'Queens Boulevard',
  zipcode: '11373',
  borough: 'Queens',
  cuisine: 'Vegetarian',
  coordinates: [ -73.876, 40.737 ],
  grades: [
    {
      date: ISODate('2024-06-18T00:00:00.000Z'),
      grade: 'A',
      score: 9
    },
    {
      date: ISODate('2023-12-08T00:00:00.000Z'),
      grade: 'B',
```

```
    score: 16
  }
]
},
{
  _id: ObjectId('68edee02344eaff523eec4b9'),
  restaurant_id: 'R1007',
  name: 'Burger Hub',
  building: '9',
  street: 'Richmond Avenue',
  zipcode: '10312',
  borough: 'Staten Island',
  cuisine: 'American',
  coordinates: [ -74.176, 40.543 ],
  grades: [
    {
      date: ISODate('2024-07-12T00:00:00.000Z'),
      grade: 'A',
      score: 6
    },
    {
      date: ISODate('2024-01-03T00:00:00.000Z'),
      grade: 'A',
      score: 9
    }
  ]
},
{
  _id: ObjectId('68edee02344eaff523eec4ba'),
```

```
restaurant_id: 'R1008',
name: 'Paris Café',
building: '105',
street: 'Broadway',
zipcode: '10006',
borough: 'Manhattan',
cuisine: 'French',
coordinates: [ -74.013, 40.707 ],
grades: [
  {
    date: ISODate('2024-08-22T00:00:00.000Z'),
    grade: 'A',
    score: 4
  },
  {
    date: ISODate('2024-02-10T00:00:00.000Z'),
    grade: 'A',
    score: 7
  }
],
{
  _id: ObjectId('68edee02344eaff523eec4bb'),
  restaurant_id: 'R1009',
  name: 'Royal Biryani',
  building: '17',
  street: 'Atlantic Avenue',
  zipcode: '11217',
  borough: 'Brooklyn',
```

```
cuisine: 'Indian',
coordinates: [ -73.977, 40.683 ],
grades: [
  {
    date: ISODate('2024-09-11T00:00:00.000Z'),
    grade: 'B',
    score: 13
  },
  {
    date: ISODate('2024-03-07T00:00:00.000Z'),
    grade: 'A',
    score: 9
  }
],
},
{
  _id: ObjectId('68edee02344eaff523eec4bc'),
  restaurant_id: 'R1010',
  name: 'Ocean Grill',
  building: '65',
  street: 'Grand Concourse',
  zipcode: '10452',
  borough: 'Bronx',
  cuisine: 'Seafood',
  coordinates: [ -73.921, 40.838 ],
  grades: [
    {
      date: ISODate('2024-10-01T00:00:00.000Z'),
      grade: 'A',
```

```
        score: 8
    },
    {
        date: ISODate('2024-04-15T00:00:00.000Z'),
        grade: 'B',
        score: 15
    }
]
}
```

// 2. Display only restaurant_id, name, borough, and cuisine fields for all documents

```
db.restaurants.find(
    {},
    { restaurant_id: 1, name: 1, borough: 1, cuisine: 1, _id: 0 }
)
```

Output :

```
[
  {
    restaurant_id: 'R1001',
    name: 'Golden Dragon',
    borough: 'Manhattan',
    cuisine: 'Chinese'
  },
  {
    restaurant_id: 'R1002',
    name: 'Spice Heaven',
    borough: 'Queens',
    cuisine: 'Indian'
  },
]
```

```
{
  restaurant_id: 'R1003',
  name: 'Bella Roma',
  borough: 'Brooklyn',
  cuisine: 'Italian'
},
{
  restaurant_id: 'R1004',
  name: 'Taco Fiesta',
  borough: 'Bronx',
  cuisine: 'Mexican'
},
{
  restaurant_id: 'R1005',
  name: 'Sushi Zen',
  borough: 'Manhattan',
  cuisine: 'Japanese'
},
{
  restaurant_id: 'R1006',
  name: 'Green Garden',
  borough: 'Queens',
  cuisine: 'Vegetarian'
},
{
  restaurant_id: 'R1007',
  name: 'Burger Hub',
  borough: 'Staten Island',
  cuisine: 'American'
}
```

```
},  
{  
  restaurant_id: 'R1008',  
  name: 'Paris Café',  
  borough: 'Manhattan',  
  cuisine: 'French'  
},  
{  
  restaurant_id: 'R1009',  
  name: 'Royal Biryani',  
  borough: 'Brooklyn',  
  cuisine: 'Indian'  
},  
{  
  restaurant_id: 'R1010',  
  name: 'Ocean Grill',  
  borough: 'Bronx',  
  cuisine: 'Seafood'  
}  
]
```


Practical 11 : 1. Write a MongoDB query to display the fields restaurant_id, name, borough and cuisine, but exclude the field _id for all the documents in the collection restaurant .

2. Write a MongoDB query to display all the restaurant which is in the borough Bronx.

```
// Use the database
```

```
use restaurantDB
```

```
// 1. Write a MongoDB query to display the fields restaurant_id, name, borough and cuisine, but exclude the field _id for all the documents in the collection restaurant .
```

```
db.restaurants.insertMany([
```

```
{
```

```
  restaurant_id: 1,
```

```
  name: "Spice Villa",
```

```
  borough: "Mumbai",
```

```
  cuisine: "Indian",
```

```
  building_number: "123",
```

```
  street_name: "MG Road",
```

```
  zip_code: "400001",
```

```
  coordinates: { longitude: 73.8567, latitude: 18.5204 },
```

```
  grades: [
```

```
    { date: ISODate("2024-01-15"), grade: "A", score: 95 }
```

```
  ]
```

```
},
```

```
{
```

```
  restaurant_id: 2,
```

```
  name: "Bronx Diner",
```

```
  borough: "Bronx",
```

```
  cuisine: "American",
```

```
  building_number: "45",
```

```
  street_name: "Fordham Road",
```

```
zip_code: "10458",
coordinates: { longitude: -73.9030, latitude: 40.8620 },
grades: [
  { date: ISODate("2023-11-12"), grade: "B", score: 85 }
],
{
```

```
  restaurant_id: 3,
  name: "Bronx Diner",
  borough: "Bronx",
  cuisine: "American",
  building_number: "45",
  street_name: "Fordham Road",
  zip_code: "10458",
  coordinates: { longitude: -73.9030, latitude: 40.8620 },
  grades: [
    { date: ISODate("2023-11-12"), grade: "B", score: 85 }
  ],
},
```

```
{
  restaurant_id: 4,
  name: "Bronx Diner",
  borough: "Queens",
  cuisine: "American",
  building_number: "45",
  street_name: "Fordham Road",
  zip_code: "10458",
  coordinates: { longitude: -73.9030, latitude: 40.8620 },
  grades: [
```

```
    { date: ISODate("2023-11-12"), grade: "B", score: 85 }
  ]
},
{
  restaurant_id: 5,
  name: "Bronx Diner",
  borough: "Bronx",
  cuisine: "American",
  building_number: "45",
  street_name: "Fordham Road",
  zip_code: "10458",
  coordinates: { longitude: -73.9030, latitude: 40.8620 },
  grades: [
    { date: ISODate("2023-11-12"), grade: "B", score: 85 }
  ]
},
{
  restaurant_id: 6,
  name: "Bronx Diner",
  borough: "Bronx",
  cuisine: "American",
  building_number: "45",
  street_name: "Fordham Road",
  zip_code: "10458",
  coordinates: { longitude: -73.9030, latitude: 40.8620 },
  grades: [
    { date: ISODate("2023-11-12"), grade: "B", score: 85 }
  ]
},
```

```
{
  restaurant_id: 7,
  name: "Bronx Diner",
  borough: "Queens",
  cuisine: "American",
  building_number: "45",
  street_name: "Fordham Road",
  zip_code: "10458",
  coordinates: { longitude: -73.9030, latitude: 40.8620 },
  grades: [
    { date: ISODate("2023-11-12"), grade: "B", score: 85 }
  ]
},
```

```
{
  restaurant_id: 8,
  name: "Bronx Diner",
  borough: "Manhattan",
  cuisine: "American",
  building_number: "45",
  street_name: "Fordham Road",
  zip_code: "10458",
  coordinates: { longitude: -73.9030, latitude: 40.8620 },
  grades: [
    { date: ISODate("2023-11-12"), grade: "B", score: 85 }
  ]
},
```

```
{
  restaurant_id: 9,
  name: "Bronx Diner",
```

```

    borough: "Bronx",
    cuisine: "American",
    building_number: "45",
    street_name: "Fordham Road",
    zip_code: "10458",
    coordinates: { longitude: -73.9030, latitude: 40.8620 },
    grades: [
      { date: ISODate("2023-11-12"), grade: "B", score: 85 }
    ]
  },
{
  restaurant_id: 10,
  name: "Bronx Diner",
  borough: "Brooklyn",
  cuisine: "American",
  building_number: "45",
  street_name: "Fordham Road",
  zip_code: "10458",
  coordinates: { longitude: -73.9030, latitude: 40.8620 },
  grades: [
    { date: ISODate("2023-11-12"), grade: "B", score: 85 }
  ]
}
])

// Display restaurant_id, name, borough, and cuisine (without _id)
db.restaurants.find(
  {},
  { restaurant_id: 1, name: 1, borough: 1, cuisine: 1, _id: 0 }
)

```

)

Output :

```
[  
  {  
    restaurant_id: 1,  
    name: 'Spice Villa',  
    borough: 'Mumbai',  
    cuisine: 'Indian'  
  },  
  {  
    restaurant_id: 2,  
    name: 'Bronx Diner',  
    borough: 'Bronx',  
    cuisine: 'American'  
  },  
  {  
    restaurant_id: 3,  
    name: 'Bronx Diner',  
    borough: 'Bronx',  
    cuisine: 'American'  
  },  
  {  
    restaurant_id: 4,  
    name: 'Bronx Diner',  
    borough: 'Queens',  
    cuisine: 'American'  
  },  
  {  
    restaurant_id: 5,
```

```
name: 'Bronx Diner',  
borough: 'Bronx',  
cuisine: 'American'
```

```
},
```

```
{
```

```
restaurant_id: 6,  
name: 'Bronx Diner',  
borough: 'Bronx',  
cuisine: 'American'
```

```
},
```

```
{
```

```
restaurant_id: 7,  
name: 'Bronx Diner',  
borough: 'Queens',  
cuisine: 'American'
```

```
},
```

```
{
```

```
restaurant_id: 8,  
name: 'Bronx Diner',  
borough: 'Manhattan',  
cuisine: 'American'
```

```
},
```

```
{
```

```
restaurant_id: 9,  
name: 'Bronx Diner',  
borough: 'Bronx',  
cuisine: 'American'
```

```
},
```

```
{
```

```
    restaurant_id: 10,  
    name: 'Bronx Diner',  
    borough: 'Brooklyn',  
    cuisine: 'American'  
  }  
]  
  
// 2. Display all restaurants in the borough "Bronx"  
db.restaurants.find({ borough: "Bronx" })
```

Output :

```
[  
  {  
    _id: ObjectId('68edf26d344eaff523eec4be'),  
    restaurant_id: 2,  
    name: 'Bronx Diner',  
    borough: 'Bronx',  
    cuisine: 'American',  
    building_number: '45',  
    street_name: 'Fordham Road',  
    zip_code: '10458',  
    coordinates: { longitude: -73.903, latitude: 40.862 },  
    grades: [  
      {  
        date: ISODate('2023-11-12T00:00:00.000Z'),  
        grade: 'B',  
        score: 85  
      }  
    ]  
  },  
  {
```



```
_id: ObjectId('68edf26d344eaff523eec4bf'),
restaurant_id: 3,
name: 'Bronx Diner',
borough: 'Bronx',
cuisine: 'American',
building_number: '45',
street_name: 'Fordham Road',
zip_code: '10458',
coordinates: { longitude: -73.903, latitude: 40.862 },
grades: [
  {
    date: ISODate('2023-11-12T00:00:00.000Z'),
    grade: 'B',
    score: 85
  }
],
},
{
```

```
_id: ObjectId('68edf26d344eaff523eec4c1'),
restaurant_id: 5,
name: 'Bronx Diner',
borough: 'Bronx',
cuisine: 'American',
building_number: '45',
street_name: 'Fordham Road',
zip_code: '10458',
coordinates: { longitude: -73.903, latitude: 40.862 },
grades: [
  {
```

```
    date: ISODate('2023-11-12T00:00:00.000Z'),
    grade: 'B',
    score: 85
  }
]
},
{
  _id: ObjectId('68edf26d344eaff523eec4c2'),
  restaurant_id: 6,
  name: 'Bronx Diner',
  borough: 'Bronx',
  cuisine: 'American',
  building_number: '45',
  street_name: 'Fordham Road',
  zip_code: '10458',
  coordinates: { longitude: -73.903, latitude: 40.862 },
  grades: [
    {
      date: ISODate('2023-11-12T00:00:00.000Z'),
      grade: 'B',
      score: 85
    }
  ]
},
{
  _id: ObjectId('68edf26d344eaff523eec4c5'),
  restaurant_id: 9,
  name: 'Bronx Diner',
  borough: 'Bronx',
```

```
cuisine: 'American',
building_number: '45',
street_name: 'Fordham Road',
zip_code: '10458',
coordinates: { longitude: -73.903, latitude: 40.862 },
grades: [
  {
    date: ISODate('2023-11-12T00:00:00.000Z'),
    grade: 'B',
    score: 85
  }
]
}
```

Practical 12 : 1. Write a MongoDB query to display the first 5 restaurants which are in the borough Bronx.

2. Write a MongoDB query to display the next 5 restaurants after skipping first 5 which are in the borough Bronx.

use restaurantDB

// Insert 10 sample restaurants in the borough "Bronx"

```
db.restaurants.insertMany([
  { restaurant_id: 1, name: "Bronx Eatery 1", borough: "Bronx", cuisine: "Indian" },
  { restaurant_id: 2, name: "Bronx Eatery 2", borough: "Bronx", cuisine: "Italian" },
  { restaurant_id: 3, name: "Bronx Eatery 3", borough: "Bronx", cuisine: "Mexican" },
  { restaurant_id: 4, name: "Bronx Eatery 4", borough: "Bronx", cuisine: "Chinese" },
  { restaurant_id: 5, name: "Bronx Eatery 5", borough: "Bronx", cuisine: "Thai" },
  { restaurant_id: 6, name: "Bronx Eatery 6", borough: "Bronx", cuisine: "Japanese" },
  { restaurant_id: 7, name: "Bronx Eatery 7", borough: "Bronx", cuisine: "Korean" },
  { restaurant_id: 8, name: "Bronx Eatery 8", borough: "Bronx", cuisine: "Greek" },
  { restaurant_id: 9, name: "Bronx Eatery 9", borough: "Bronx", cuisine: "French" },
  { restaurant_id: 10, name: "Bronx Eatery 10", borough: "Bronx", cuisine: "American" }
])
```

// 1. Display the first 5 restaurants in the borough "Bronx"

```
db.restaurants.find({ borough: "Bronx" }).limit(5)
```

Output :

```
[
  {
    _id: ObjectId('68ea548b613a3c20faecc4b0'),
    restaurant_id: 1,
    name: 'Bronx Eatery 1',
    borough: 'Bronx',
```

```
cuisine: 'Indian'
},
{
  _id: ObjectId('68ea548b613a3c20faeec4b1'),
  restaurant_id: 2,
  name: 'Bronx Eatery 2',
  borough: 'Bronx',
  cuisine: 'Italian'
},
{
  _id: ObjectId('68ea548b613a3c20faeec4b2'),
  restaurant_id: 3,
  name: 'Bronx Eatery 3',
  borough: 'Bronx',
  cuisine: 'Mexican'
},
{
  _id: ObjectId('68ea548b613a3c20faeec4b3'),
  restaurant_id: 4,
  name: 'Bronx Eatery 4',
  borough: 'Bronx',
  cuisine: 'Chinese'
},
{
  _id: ObjectId('68ea548b613a3c20faeec4b4'),
  restaurant_id: 5,
  name: 'Bronx Eatery 5',
  borough: 'Bronx',
  cuisine: 'Thai'
```

```
}  
]
```

// 2. Display the next 5 restaurants in the borough "Bronx" (skip first 5)

```
db.restaurants.find({ borough: "Bronx" }).skip(5).limit(5)
```

Output :

```
[  
  {  
    _id: ObjectId('68ea548b613a3c20faeec4b5'),  
    restaurant_id: 6,  
    name: 'Bronx Eatery 6',  
    borough: 'Bronx',  
    cuisine: 'Japanese'  
  },  
  {  
    _id: ObjectId('68ea548b613a3c20faeec4b6'),  
    restaurant_id: 7,  
    name: 'Bronx Eatery 7',  
    borough: 'Bronx',  
    cuisine: 'Korean'  
  },  
  {  
    _id: ObjectId('68ea548b613a3c20faeec4b7'),  
    restaurant_id: 8,  
    name: 'Bronx Eatery 8',  
    borough: 'Bronx',  
    cuisine: 'Greek'  
  },  
]
```

```
{
  _id: ObjectId('68ea548b613a3c20faeec4b8'),
  restaurant_id: 9,
  name: 'Bronx Eatery 9',
  borough: 'Bronx',
  cuisine: 'French'
},
{
  _id: ObjectId('68ea548b613a3c20faeec4b9'),
  restaurant_id: 10,
  name: 'Bronx Eatery 10',
  borough: 'Bronx',
  cuisine: 'American'
}
]
```

Practical 13 : 1. Write a MongoDB query to find the restaurants who achieved a score more than 90 .

2. Write a MongoDB query to find the restaurants that achieved a score, more than 80 but less than 100.

```
// Use the database
```

```
use restaurantDB
```

```
// Insert 3 sample records into restaurants collection
```

```
db.restaurants.insertMany([
```

```
  { restaurant_id: 1, name: "Spice Villa", borough: "Mumbai", cuisine: "Indian", grades: [ {  
    date: "2024-01-15", grade: "A", score: 92 } ] },
```

```
  { restaurant_id: 2, name: "Bronx Diner", borough: "Bronx", cuisine: "American", grades: [ {  
    date: "2024-03-12", grade: "B", score: 85 } ] },
```

```
  { restaurant_id: 3, name: "Green Garden", borough: "Delhi", cuisine: "Chinese", grades: [ {  
    date: "2024-05-10", grade: "A", score: 78 } ] },
```

```
  { restaurant_id: 4, name: "Spice Villa", borough: "Bronx", cuisine: "American", grades: [ {  
    date: "2024-04-11", grade: "B", score: 85 } ] },
```

```
  { restaurant_id: 5, name: "Paris Café", borough: "Queens", cuisine: "American", grades: [ {  
    date: "2024-05-21", grade: "B", score: 93 } ] },
```

```
  { restaurant_id: 6, name: "Green Garden", borough: "Bronx", cuisine: "Seafood", grades: [ {  
    date: "2024-02-12", grade: "B", score: 99 } ] },
```

```
  { restaurant_id: 7, name: "Bronx Diner", borough: "Manhattan", cuisine: "American", grades:  
  [ { date: "2024-01-22", grade: "B", score: 85 } ] },
```

```
  { restaurant_id: 8, name: "Spice Villa", borough: "Bronx", cuisine: "Indian", grades: [ { date:  
    "2024-03-12", grade: "B", score: 85 } ] },
```

```
  { restaurant_id: 9, name: "Paris Café", borough: "Manhattan", cuisine: "American", grades: [  
    { date: "2024-06-15", grade: "B", score: 85 } ] },
```

```
  { restaurant_id: 10, name: "Green Garden", borough: "Queens", cuisine: "French", grades: [ {  
    date: "2024-03-19", grade: "B", score: 72 } ] }  
]
```

```
// 1. Find restaurants with score greater than 90
```

```
db.restaurants.find({ "grades.score": { $gt: 90 } })
```


Output :

```
[
  {
    _id: ObjectId('68edf5c8344eaff523eec4c7'),
    restaurant_id: 1,
    name: 'Spice Villa',
    borough: 'Mumbai',
    cuisine: 'Indian',
    grades: [ { date: '2024-01-15', grade: 'A', score: 92 } ]
  },
  {
    _id: ObjectId('68edf5c8344eaff523eec4cb'),
    restaurant_id: 5,
    name: 'Paris Café',
    borough: 'Queens',
    cuisine: 'American',
    grades: [ { date: '2024-05-21', grade: 'B', score: 93 } ]
  },
  {
    _id: ObjectId('68edf5c8344eaff523eec4cc'),
    restaurant_id: 6,
    name: 'Green Garden',
    borough: 'Bronx',
    cuisine: 'Seafood',
    grades: [ { date: '2024-02-12', grade: 'B', score: 99 } ]
  }
]
```

// 2. Find restaurants with score more than 80 but less than 100

```
db.restaurants.find( { "grades.score": { $gt: 80, $lt: 100 } })
```

Output :

```
[
  {
    _id: ObjectId('68edf5c8344eaff523eec4c7'),
    restaurant_id: 1,
    name: 'Spice Villa',
    borough: 'Mumbai',
    cuisine: 'Indian',
    grades: [ { date: '2024-01-15', grade: 'A', score: 92 } ]
  },
  {
    _id: ObjectId('68edf5c8344eaff523eec4c8'),
    restaurant_id: 2,
    name: 'Bronx Diner',
    borough: 'Bronx',
    cuisine: 'American',
    grades: [ { date: '2024-03-12', grade: 'B', score: 85 } ]
  },
  {
    _id: ObjectId('68edf5c8344eaff523eec4ca'),
    restaurant_id: 4,
    name: 'Spice Villa',
    borough: 'Bronx',
    cuisine: 'American',
    grades: [ { date: '2024-04-11', grade: 'B', score: 85 } ]
  },
]
```

```
{
  _id: ObjectId('68edf5c8344eaff523eec4cb'),
  restaurant_id: 5,
  name: 'Paris Café',
  borough: 'Queens',
  cuisine: 'American',
  grades: [ { date: '2024-05-21', grade: 'B', score: 93 } ]
},
```

```
{
  _id: ObjectId('68edf5c8344eaff523eec4cc'),
  restaurant_id: 6,
  name: 'Green Garden',
  borough: 'Bronx',
  cuisine: 'Seafood',
  grades: [ { date: '2024-02-12', grade: 'B', score: 99 } ]
},
```

```
{
  _id: ObjectId('68edf5c8344eaff523eec4cd'),
  restaurant_id: 7,
  name: 'Bronx Diner',
  borough: 'Manhattan',
  cuisine: 'American',
  grades: [ { date: '2024-01-22', grade: 'B', score: 85 } ]
},
```

```
{
  _id: ObjectId('68edf5c8344eaff523eec4ce'),
  restaurant_id: 8,
  name: 'Spice Villa',
  borough: 'Bronx',
```

```
cuisine: 'Indian',
grades: [ { date: '2024-03-12', grade: 'B', score: 85 } ]
},
{
  _id: ObjectId('68edf5c8344eaff523eec4cf'),
  restaurant_id: 9,
  name: 'Paris Café',
  borough: 'Manhattan',
  cuisine: 'American',
  grades: [ { date: '2024-06-15', grade: 'B', score: 85 } ]
}
]
```

Practical 14 : Write a MongoDB query to find the restaurants which do not prepare any cuisine of 'American ' and achieved a grade point 'A' not belonging to the boroughBrooklyn. The document must be displayed according to the cuisine in descending order.

```
// Use database
```

```
use restaurantDB
```

```
// Insert sample records
```

```
db.restaurants.insertMany([
```

```
  { restaurant_id: 1, name: "Spice Villa", borough: "Mumbai", cuisine: "Indian", grades: [ {  
    date: "2024-01-15", grade: "A", score: 92 } ] },
```

```
  { restaurant_id: 2, name: "Bronx Diner", borough: "Bronx", cuisine: "American", grades: [ {  
    date: "2024-03-12", grade: "A", score: 95 } ] },
```

```
  { restaurant_id: 3, name: "Green Garden", borough: "Delhi", cuisine: "Chinese", grades: [ {  
    date: "2024-05-10", grade: "A", score: 88 } ] },
```

```
  { restaurant_id: 4, name: "Tandoori Flames", borough: "Brooklyn", cuisine: "Indian",  
    grades: [ { date: "2024-04-10", grade: "A", score: 90 } ] },
```

```
  { restaurant_id: 5, name: "Veggie Delight", borough: "Pune", cuisine: "Vegetarian", grades:  
    [ { date: "2024-02-20", grade: "B", score: 80 } ] },
```

```
  { restaurant_id: 6, name: "Bronx Diner", borough: "Nashik", cuisine: "Indian", grades: [ {  
    date: "2024-05-30", grade: "A", score: 93 } ] },
```

```
  { restaurant_id: 7, name: "Bronx Diner", borough: "Bronx", cuisine: "American", grades: [ {  
    date: "2024-06-18", grade: "A", score: 95 } ] },
```

```
  { restaurant_id: 8, name: "Green Garden", borough: "Delhi", cuisine: "Chinese", grades: [ {  
    date: "2024-08-14", grade: "B", score: 89 } ] },
```

```
  { restaurant_id: 9, name: "Tandoori Flames", borough: "Brooklyn", cuisine: "Indian",  
    grades: [ { date: "2024-06-11", grade: "A", score: 91 } ] },
```

```
  { restaurant_id: 10, name: "Veggie Delight", borough: "Pune", cuisine: "American", grades:  
    [ { date: "2024-02-22", grade: "B", score: 80 } ] }  
])
```

```
// Query to find restaurants:
```

```
db.restaurants.find({
  cuisine: { $ne: "American" },
  "grades.grade": "A",
  borough: { $ne: "Brooklyn" }
}).sort({ cuisine: -1 })
```

Output :

```
[
  {
    _id: ObjectId('68edf85a344eaff523eec4d1'),
    restaurant_id: 1,
    name: 'Spice Villa',
    borough: 'Mumbai',
    cuisine: 'Indian',
    grades: [ { date: '2024-01-15', grade: 'A', score: 92 } ]
  },
  {
    _id: ObjectId('68edf85a344eaff523eec4d6'),
    restaurant_id: 6,
    name: 'Bronx Diner',
    borough: 'Nashik',
    cuisine: 'Indian',
    grades: [ { date: '2024-05-30', grade: 'A', score: 93 } ]
  },
  {
    _id: ObjectId('68edf85a344eaff523eec4d3'),
    restaurant_id: 3,
    name: 'Green Garden',
    borough: 'Delhi',
```

cuisine: 'Chinese',

grades: [{ date: '2024-05-10', grade: 'A', score: 88 }]

}

]

Practical 15 : Write a MongoDB query to find the restaurant Id,name, borough and cuisine for those restaurants which contain 'Wil' as first three letters for its name.

```
// Use database
```

```
use restaurantDB
```

```
// Insert sample records
```

```
db.restaurants.insertMany([
```

```
  { restaurant_id: 1, name: "Wild Bites", borough: "Mumbai", cuisine: "Indian", grades: [ {  
    date: "2024-01-10", grade: "A", score: 90 } ] },
```

```
  { restaurant_id: 2, name: "Wilson Grill", borough: "Delhi", cuisine: "Continental", grades: [  
    { date: "2024-02-15", grade: "B", score: 85 } ] },
```

```
  { restaurant_id: 3, name: "Bella Casa", borough: "Pune", cuisine: "Italian", grades: [ { date:  
    "2024-03-20", grade: "A", score: 88 } ] },
```

```
  { restaurant_id: 4, name: "Willow Tree", borough: "Bangalore", cuisine: "Chinese", grades:  
    [ { date: "2024-04-25", grade: "A", score: 92 } ] },
```

```
  { restaurant_id: 5, name: "Willow Tree", borough: "Bangalore", cuisine: "Chinese", grades: [  
    { date: "2024-04-25", grade: "A", score: 92 } ] },
```

```
  { restaurant_id: 6, name: "Willow Tree", borough: "Bangalore", cuisine: "American", grades:  
    [ { date: "2024-06-25", grade: "A", score: 92 } ] },
```

```
  { restaurant_id: 7, name: "Taco Fiesta", borough: "Pune", cuisine: "Indian", grades: [ { date:  
    "2024-03-22", grade: "B", score: 80 } ] },
```

```
  { restaurant_id: 8, name: "Pasta Palace", borough: "Nashik", cuisine: "Chinese", grades: [ {  
    date: "2024-08-21", grade: "A", score: 81 } ] },
```

```
  { restaurant_id: 9, name: "Willow Tree", borough: "Bangalore", cuisine: "Italian", grades: [ {  
    date: "2024-06-20", grade: "B", score: 90 } ] },
```

```
  { restaurant_id: 10, name: "Bronx Deli", borough: "Mumbai", cuisine: "Chinese", grades: [ {  
    date: "2024-02-24", grade: "A", score: 91 } ] }
```

```
])
```


// Query to find restaurants with names starting with "Wil"

```
db.restaurants.find(  
  { name: { $regex: /^Wil/ } },  
  { restaurant_id: 1, name: 1, borough: 1, cuisine: 1, _id: 0 }  
)
```

Output :

```
[  
  {  
    restaurant_id: 1,  
    name: 'Wild Bites',  
    borough: 'Mumbai',  
    cuisine: 'Indian'  
  },  
  {  
    restaurant_id: 2,  
    name: 'Wilson Grill',  
    borough: 'Delhi',  
    cuisine: 'Continental'  
  },  
  {  
    restaurant_id: 4,  
    name: 'Willow Tree',  
    borough: 'Bangalore',  
    cuisine: 'Chinese'  
  },  
  {  
    restaurant_id: 5,  
    name: 'Willow Tree',
```

```
    borough: 'Bangalore',  
    cuisine: 'Chinese'  
  },  
  {  
    restaurant_id: 6,  
    name: 'Willow Tree',  
    borough: 'Bangalore',  
    cuisine: 'American'  
  },  
  {  
    restaurant_id: 9,  
    name: 'Willow Tree',  
    borough: 'Bangalore',  
    cuisine: 'Italian'  
  }  
]
```

Practical 16 : Write a MongoDB query to find the restaurant Id,name, borough and cuisine for those restaurants which contain 'ces' as the last three letters for its name.

```
// Use database
```

```
use restaurantDB
```

```
// Insert sample records
```

```
db.restaurants.insertMany([
```

```
{ restaurant_id: 1, name: "Spices", borough: "Mumbai", cuisine: "Indian", grades: [ { date: "2024-01-10", grade: "A", score: 90 } ] },
```

```
{ restaurant_id: 2, name: "Delices", borough: "Brooklyn", cuisine: "French", grades: [ { date: "2024-03-15", grade: "B", score: 85 } ] },
```

```
{ restaurant_id: 3, name: " Willow Tree", borough: "Queens", cuisine: "Italian", grades: [ { date: "2024-05-20", grade: "A", score: 92 } ] },
```

```
{ restaurant_id: 4, name: "Palaces", borough: "Bronx", cuisine: "Chinese", grades: [ { date: "2024-02-05", grade: "A", score: 88 } ] },
```

```
{ restaurant_id: 5, name: " Taco Fiesta ", borough: "Manhattan", cuisine: "Dessert", grades: [ { date: "2024-01-25", grade: "B", score: 80 } ] },
```

```
{ restaurant_id: 6, name: "Bounces", borough: "Delhi", cuisine: "Fast Food", grades: [ { date: "2024-06-12", grade: "A", score: 91 } ] },
```

```
{ restaurant_id: 7, name: " Willow Tree", borough: "Brooklyn", cuisine: "American", grades: [ { date: "2024-04-10", grade: "B", score: 84 } ] },
```

```
{ restaurant_id: 8, name: "Services", borough: "Queens", cuisine: "Mexican", grades: [ { date: "2024-05-05", grade: "A", score: 89 } ] },
```

```
{ restaurant_id: 9, name: " Taco Fiesta ", borough: "Bronx", cuisine: "Thai", grades: [ { date: "2024-03-20", grade: "A", score: 93 } ] },
```

```
{ restaurant_id: 10, name: "Graces", borough: "Manhattan", cuisine: "Continental", grades: [ { date: "2024-02-15", grade: "A", score: 95 } ] }
```

```
])
```

```
// Query to find restaurants with names ending with "ces"
```

```
db.restaurants.find(
```

```
{ name: { $regex: /ces$/ } },  
{ restaurant_id: 1, name: 1, borough: 1, cuisine: 1, _id: 0 }  
)
```

Output :

```
{  
  restaurant_id: 1,  
  name: 'Spices',  
  borough: 'Mumbai',  
  cuisine: 'Indian'  
},  
{  
  restaurant_id: 2,  
  name: 'Delices',  
  borough: 'Brooklyn',  
  cuisine: 'French'  
},  
{  
  restaurant_id: 4,  
  name: 'Palaces',  
  borough: 'Bronx',  
  cuisine: 'Chinese'  
},  
{  
  restaurant_id: 6,  
  name: 'Bounces',  
  borough: 'Delhi',  
  cuisine: 'Fast Food'  
},  
{
```

```
    restaurant_id: 8,  
    name: 'Services',  
    borough: 'Queens',  
    cuisine: 'Mexican'  
  },  
  {  
    restaurant_id: 10,  
    name: 'Graces',  
    borough: 'Manhattan',  
    cuisine: 'Continental'  
  }  
]
```

Practical 17 : Write a MongoDB query to find the restaurant Id,name, borough and cuisine for those restaurants which contain 'Reg' as three letters somewhere in its name.

```
// Use database
```

```
use restaurantDB
```

```
// Insert sample records
```

```
db.restaurants.insertMany([
```

```
  { restaurant_id: 1, name: "Regal Spice", borough: "Delhi", cuisine: "Indian", grades: [ {  
    date: "2024-01-15", grade: "A", score: 90 } ] },
```

```
  { restaurant_id: 2, name: " Palaces ", borough: "Mumbai", cuisine: "Continental", grades: [ {  
    date: "2024-03-10", grade: "B", score: 85 } ] },
```

```
  { restaurant_id: 3, name: "Royal Regale", borough: "Kolkata", cuisine: "Italian", grades: [ {  
    date: "2024-05-12", grade: "A", score: 91 } ] },
```

```
  { restaurant_id: 4, name: " Graces ", borough: "Bangalore", cuisine: "Mexican", grades: [ {  
    date: "2024-04-25", grade: "A", score: 92 } ] },
```

```
  { restaurant_id: 5, name: "Regatta Dine", borough: "Pune", cuisine: "Seafood", grades: [ {  
    date: "2024-02-14", grade: "A", score: 88 } ] },
```

```
  { restaurant_id: 6, name: " Palaces ", borough: "Hyderabad", cuisine: "North Indian",  
    grades: [ { date: "2024-06-01", grade: "B", score: 84 } ] },
```

```
  { restaurant_id: 7, name: "Evergreen Bites", borough: "Chennai", cuisine: "Vegetarian",  
    grades: [ { date: "2024-07-10", grade: "A", score: 89 } ] },
```

```
  { restaurant_id: 8, name: "Regency Palace", borough: "Delhi", cuisine: "Chinese", grades: [  
    { date: "2024-08-20", grade: "A", score: 93 } ] },
```

```
  { restaurant_id: 9, name: "Regional Flavors", borough: "Ahmedabad", cuisine: "Gujarati",  
    grades: [ { date: "2024-09-05", grade: "A", score: 90 } ] },
```

```
  { restaurant_id: 10, name: " Graces ", borough: "Jaipur", cuisine: "Cafe", grades: [ { date:  
    "2024-10-02", grade: "B", score: 83 } ] }  
])
```

```
// Query to find restaurants with 'Reg' in the name
```

```
db.restaurants.find(
```

```
  { name: { $regex: /Reg/ } },
```

```
  { restaurant_id: 1, name: 1, borough: 1, cuisine: 1, _id: 0 }
```

)

Output :

```
{
  restaurant_id: 1,
  name: 'Regal Spice',
  borough: 'Delhi',
  cuisine: 'Indian'
},
{
  restaurant_id: 3,
  name: 'Royal Regale',
  borough: 'Kolkata',
  cuisine: 'Italian'
},
{
  restaurant_id: 5,
  name: 'Regatta Dine',
  borough: 'Pune',
  cuisine: 'Seafood'
},
{
  restaurant_id: 8,
  name: 'Regency Palace',
  borough: 'Delhi',
  cuisine: 'Chinese'
},
{
  restaurant_id: 9,
```

```
    name: 'Regional Flavors',  
    borough: 'Ahmedabad',  
    cuisine: 'Gujarati'  
  }  
]
```


Practical 18 : Write a MongoDB query to find the restaurants which belong to the borough Bronx and prepared either American or Chinese dish.

```
// Use database

use restaurantDB

// Insert sample records

db.restaurants.insertMany([

  {name:"Bronx Diner",borough:"Bronx",cuisine:"American"},

  {name:"Dragon Palace",borough:"Bronx",cuisine:"Chinese"},

  {name:"Pizza King",borough:"Bronx",cuisine:"Italian"},

  {name:"Tandoori spice",borough:"Manhatta",cuisine:"Indian"},

  {name:"Burger Town",borough:"Bronx",cuisine:"American"},

  {name:"Great Wall",borough:" Bronx ",cuisine:"Chinese"},

  {name:" Burger Town ",borough:"Queens",cuisine:" Indian "},

  {name:"Great Wall",borough:"Queens",cuisine:"Chinese"},

  {name:" Tandoori spice ",borough:" Manhatta ",cuisine:" Indian "},

  {name:"Great Wall",borough:"Queens",cuisine:"Chinese"}

]);

// Find restaurants in Bronx serving American or Chinese cuisine (all fields shown)

db.restaurants.find(

  {

    borough: "Bronx",

    cuisine: { $in: ["American", "Chinese"] }

  }

)
```

Output :

```
[

  {

    _id: ObjectId('68ee01c2344eaff523eec517'),
```

```
    name: 'Bronx Diner',
    borough: 'Bronx',
    cuisine: 'American'
  },
  {
    _id: ObjectId('68ee01c2344eaff523eec518'),
    name: 'Dragon Palace',
    borough: 'Bronx',
    cuisine: 'Chinese'
  },
  {
    _id: ObjectId('68ee01c2344eaff523eec51b'),
    name: 'Burger Town',
    borough: 'Bronx',
    cuisine: 'American'
  }
]
```

Practical 19 : Write a MongoDB query to find the restaurant Id,name, borough and cuisine for those restaurants which belong to the borough Staten Island or Queens or Bronx or Brooklyn.

```
// Use database
```

```
use restaurantDB
```

```
// Insert sample records
```

```
db.restaurants.insertMany([
```

```
  { restaurant_id: 1, name: "Bronx Diner", borough: " Queens ", cuisine: "American" },
```

```
  { restaurant_id: 2, name: "Queens Grill", borough: "Queens", cuisine: "Chinese" },
```

```
  { restaurant_id: 3, name: "Brooklyn Pizza", borough: "Brooklyn", cuisine: "Italian" },
```

```
  { restaurant_id: 4, name: "Staten Island Eatery", borough: "Staten Island", cuisine: "Mexican" },
```

```
  { restaurant_id: 5, name: " Burger Town", borough: "Manhattan", cuisine: "American" },
```

```
  { restaurant_id: 6, name: "Manhattan Deli", borough: " Queens ", cuisine: " Italian " },
```

```
  { restaurant_id: 7, name: " Dragon Palace", borough: " Brooklyn ", cuisine: "Indian" },
```

```
  { restaurant_id: 8, name: " Staten Island Eatery ", borough: "Manhattan", cuisine: "American" },
```

```
  { restaurant_id: 9, name: " Staten Island Eatery ", borough: " Queens ", cuisine: "American" },
```

```
  { restaurant_id: 10, name: "Manhattan Deli", borough: " Brooklyn ", cuisine: "American" }
```

```
])
```

// Query: Find restaurants in Staten Island, Queens, Bronx or Brooklyn with specific fields

```
db.restaurants.find(
```

```
  { borough: { $in: ["Staten Island", "Queens", "Bronx", "Brooklyn"] } },
```

```
  { restaurant_id: 1, name: 1, borough: 1, cuisine: 1, _id: 0 }
```

```
)
```

Output :

```
[
```

```
{
  restaurant_id: 2,
  name: 'Queens Grill',
  borough: 'Queens',
  cuisine: 'Chinese'
},
{
  restaurant_id: 3,
  name: 'Brooklyn Pizza',
  borough: 'Brooklyn',
  cuisine: 'Italian'
},
{
  restaurant_id: 4,
  name: 'Staten Island Eatery',
  borough: 'Staten Island',
  cuisine: 'Mexican'
}
]
```

Practical 20 : Write a MongoDB query to find the restaurant Id,name, borough and cuisine for those restaurants which are not belonging to the borough Staten Island Or Queens or Bronx or Brooklyn.

```
// Use database
use restaurantDB

// Insert sample records
db.restaurants.insertMany([
  { restaurant_id: 1, name: "Bronx Diner", borough: "Bronx", cuisine: "American" },
  { restaurant_id: 2, name: "Queens Grill", borough: "Queens", cuisine: "Chinese" },
  { restaurant_id: 3, name: "Brooklyn Pizza", borough: "Brooklyn", cuisine: "Italian" },
  { restaurant_id: 4, name: "Staten Island Eatery", borough: "Staten Island", cuisine:
"Mexican" },
  { restaurant_id: 5, name: "Manhattan Deli", borough: "Manhattan", cuisine: "American" },
  { restaurant_id: 6, name: " Brooklyn Pizza ", borough: " Queens ", cuisine: " Chinese " },
  { restaurant_id: 7, name: "Harlem Cafe", borough: "Harlem", cuisine: "Caribbean" },
  { restaurant_id: 8, name: "Harlem Cafe", borough: "Harlem", cuisine: "Caribbean" },
  { restaurant_id: 9, name: " Brooklyn Pizza ", borough: " Queens ", cuisine: " Chinese " },
  { restaurant_id: 10, name: "Harlem Cafe", borough: "Harlem", cuisine: "Caribbean" }
])

// Query: Find restaurants NOT in Staten Island, Queens, Bronx, or Brooklyn
db.restaurants.find(
  { borough: { $nin: ["Staten Island", "Queens", "Bronx", "Brooklyn"] } },
  { restaurant_id: 1, name: 1, borough: 1, cuisine: 1, _id: 0 }
)
```

Output :

```
[
  {
```

```
restaurant_id: 5,  
name: 'Manhattan Deli',  
borough: 'Manhattan',  
cuisine: 'American'  
},  
{  
restaurant_id: 6,  
name: ' Brooklyn Pizza ',  
borough: ' Queens ',  
cuisine: ' Chinese '  
},  
{  
restaurant_id: 7,  
name: 'Harlem Cafe',  
borough: 'Harlem',  
cuisine: 'Caribbean'  
},  
{  
restaurant_id: 8,  
name: 'Harlem Cafe',  
borough: 'Harlem',  
cuisine: 'Caribbean'  
},  
{  
restaurant_id: 9,  
name: ' Brooklyn Pizza ',  
borough: ' Queens ',  
cuisine: ' Chinese '  
},
```

```
{  
  restaurant_id: 10,  
  name: 'Harlem Cafe',  
  borough: 'Harlem',  
  cuisine: 'Caribbean'  
}  
]
```

Practical 21 : Write a MongoDB query to find the restaurant Id,name, borough and cuisine for those restaurants which achieved a score which is not more than 10.

```
// Use database
```

```
use restaurantDB
```

```
// Insert sample data
```

```
db.restaurants.insertMany([
```

```
{
```

```
  restaurant_id: 1,
```

```
  name: "Sunny Side Cafe",
```

```
  borough: "Manhattan",
```

```
  cuisine: "American",
```

```
  grades: [
```

```
    { date: new Date("2023-01-10"), grade: "A", score: 8 },
```

```
    { date: new Date("2022-11-15"), grade: "B", score: 12 } 
```

```
  ]
```

```
},
```

```
{
```

```
  restaurant_id: 2,
```

```
  name: "Ocean Breeze",
```

```
  borough: "Brooklyn",
```

```
  cuisine: "Seafood",
```

```
  grades: [
```

```
    { date: new Date("2023-03-05"), grade: "A", score: 15 },
```

```
    { date: new Date("2022-12-22"), grade: "A", score: 9 } 
```

```
  ]
```

```
},
```

```
{
```

```
  restaurant_id: 3,
```



```
name: "Green Garden",
borough: "Queens",
cuisine: "Vegetarian",
grades: [
  { date: new Date("2023-02-18"), grade: "C", score: 11 },
  { date: new Date("2022-10-10"), grade: "B", score: 7 }
],
{
  restaurant_id: 4,
  name: "Harlem Cafe",
  borough: "Manhattan",
  cuisine: "Indian",
  grades: [
    { date: new Date("2023-01-11"), grade: "A", score: 11 },
    { date: new Date("2022-11-15"), grade: "B", score: 12 }
  ],
},
{
  restaurant_id: 5,
  name: "Brooklyn Pizza ",
  borough: " Queens ",
  cuisine: "Italian",
  grades: [
    { date: new Date("2023-01-22"), grade: "A", score: 11 },
    { date: new Date("2022-11-15"), grade: "B", score: 12 }
  ],
},
{
```

```
restaurant_id: 6,  
name: "Manhattan Deli",  
borough: "Queens",  
cuisine: "American",  
grades: [  
  { date: new Date("2023-01-14"), grade: "A", score: 6 },  
  { date: new Date("2022-11-15"), grade: "B", score: 10 }  
]  
,  
{  
  restaurant_id: 7,  
  name: "Brooklyn Pizza",  
  borough: "Manhattan",  
  cuisine: "Chinese",  
  grades: [  
    { date: new Date("2023-01-10"), grade: "A", score: 10 },  
    { date: new Date("2022-11-15"), grade: "B", score: 12 }  
  ]  
},  
{  
  restaurant_id: 8,  
  name: "Bronx Diner",  
  borough: "Manhattan",  
  cuisine: "American",  
  grades: [  
    { date: new Date("2023-01-10"), grade: "A", score: 6 },  
    { date: new Date("2022-11-15"), grade: "B", score: 12 }  
  ]  
},
```

```

{
  restaurant_id: 9,
  name: "Staten Island Eatery",
  borough: "Queens",
  cuisine: "Indian",
  grades: [
    { date: new Date("2023-01-13"), grade: "A", score: 17 },
    { date: new Date("2022-11-15"), grade: "B", score: 12 }
  ]
},
{
  restaurant_id: 10,
  name: "Sunny Side Cafe",
  borough: "Manhattan",
  cuisine: "American",
  grades: [
    { date: new Date("2023-01-10"), grade: "A", score: 18 },
    { date: new Date("2022-11-15"), grade: "B", score: 17 }
  ]
}

```

)

// Query: Find restaurants with any score not more than 10

```

db.restaurants.find(
  { "grades.score": { $lte: 10 } },
  { restaurant_id: 1, name: 1, borough: 1, cuisine: 1, _id: 0 }
)

```

Output :

```
[
  {
    restaurant_id: 1,
    name: 'Sunny Side Cafe',
    borough: 'Manhattan',
    cuisine: 'American'
  },
  {
    restaurant_id: 2,
    name: 'Ocean Breeze',
    borough: 'Brooklyn',
    cuisine: 'Seafood'
  },
  {
    restaurant_id: 3,
    name: 'Green Garden',
    borough: 'Queens',
    cuisine: 'Vegetarian'
  },
  {
    restaurant_id: 6,
    name: 'Manhattan Deli',
    borough: 'Queens',
    cuisine: 'American'
  },
  {
    restaurant_id: 7,
    name: 'Brooklyn Pizza',
```

```
    borough: 'Manhattan',  
    cuisine: 'Chinese'  
  },  
  {  
    restaurant_id: 8,  
    name: 'Bronx Diner',  
    borough: 'Manhattan',  
    cuisine: 'American'  
  }  
]
```

Practical 22 : Write a MongoDB query to find the restaurant Id,name, borough and cuisine for those restaurants which prepared dish except 'American' and 'Chinese' or restaurant's name begins with letter 'Wil'.

```
// Use database
```

```
use restaurantDB
```

```
// Insert sample data
```

```
db.restaurants.insertMany([
  { restaurant_id: 1, name: "Wilson's Diner", borough: "Manhattan", cuisine: "American" },
  { restaurant_id: 2, name: "Green Garden", borough: "Queens", cuisine: "Vegetarian" },
  { restaurant_id: 3, name: "Spice Hub", borough: "Brooklyn", cuisine: "Indian" },
  { restaurant_id: 4, name: "China Palace", borough: "Bronx", cuisine: "Chinese" },
  { restaurant_id: 5, name: "Wild Orchid", borough: "Manhattan", cuisine: "Thai" },
  { restaurant_id: 6, name: "Brooklyn Pizza", borough: "Manhattan", cuisine: "American" },
  { restaurant_id: 7, name: "Green Garden", borough: "Queens", cuisine: "Indian" },
  { restaurant_id: 8, name: "Spice Hub", borough: "Brooklyn", cuisine: "Italian" },
  { restaurant_id: 9, name: "China Palace", borough: "Bronx", cuisine: "Chinese" },
  { restaurant_id: 10, name: "Brooklyn Pizza", borough: "Manhattan", cuisine: "Indian" }
])
```

// 1. Find restaurants which do NOT prepare 'American' or 'Chinese' cuisine:

```
db.restaurants.find(
  { cuisine: { $nin: ["American", "Chinese"] } },
  { restaurant_id: 1, name: 1, borough: 1, cuisine: 1, _id: 0 }
)
```

Output :

```
[
  {
    restaurant_id: 2,
    name: 'Green Garden',
```

```
    borough: 'Queens',  
    cuisine: 'Vegetarian'  
  },  
  {  
    restaurant_id: 3,  
    name: 'Spice Hub',  
    borough: 'Brooklyn',  
    cuisine: 'Indian'  
  },  
  {  
    restaurant_id: 5,  
    name: 'Wild Orchid',  
    borough: 'Manhattan',  
    cuisine: 'Thai'  
  },  
  {  
    restaurant_id: 7,  
    name: 'Green Garden',  
    borough: 'Queens',  
    cuisine: 'Indian'  
  },  
  {  
    restaurant_id: 8,  
    name: 'Spice Hub',  
    borough: 'Brooklyn',  
    cuisine: 'Italian'  
  },  
  {  
    restaurant_id: 10,
```

```
    name: 'Brooklyn Pizza',
    borough: 'Manhattan',
    cuisine: 'Indian'
  }
]
```

// 2. Find restaurants whose name starts with 'Wil':

```
db.restaurants.find(
  { name: /^Wil/ },
  { restaurant_id: 1, name: 1, borough: 1, cuisine: 1, _id: 0 }
)
```

Output :

```
[
  {
    restaurant_id: 1,
    name: "Wilson's Diner",
    borough: 'Manhattan',
    cuisine: 'American'
  },
  {
    restaurant_id: 5,
    name: 'Wild Orchid',
    borough: 'Manhattan',
    cuisine: 'Thai' } ]
```


Practical 23 : Write a MongoDB query to arrange the name of the restaurants in descending along with all the columns.

```
// Use database
use restaurantDB

// Insert sample data
db.restaurants.insertMany([
  { restaurant_id: 1, name: "Wilson's Diner", borough: "Manhattan", cuisine: "American" },
  { restaurant_id: 2, name: "Green Garden", borough: "Queens", cuisine: "Vegetarian" },
  { restaurant_id: 3, name: "Harlem Cafe", borough: "Brooklyn", cuisine: "Indian" },
  { restaurant_id: 4, name: "China Palace", borough: "Bronx", cuisine: "Chinese" },
  { restaurant_id: 5, name: "Spice Hub", borough: "Manhattan", cuisine: "Thai" },
  { restaurant_id: 6, name: "Manhattan Deli", borough: "Manhattan", cuisine: "American" },
  { restaurant_id: 7, name: "Ocean Breeze", borough: "Queens", cuisine: "Vegetarian" },
  { restaurant_id: 8, name: "Brooklyn Pizza", borough: "Brooklyn", cuisine: "Indian" },
  { restaurant_id: 9, name: "Sunny Side Cafe", borough: "Bronx", cuisine: "Chinese" },
  { restaurant_id: 10, name: "Bronx Diner", borough: "Manhattan", cuisine: "Thai" }
])

// Query to sort restaurants by name in descending order with all columns
db.restaurants.find().sort({ name: -1 })
```

Output :

```
[
  {
    _id: ObjectId('68ee0e0d344eaff523ecc567'),
    restaurant_id: 1,
    name: "Wilson's Diner",
    borough: 'Manhattan',
    cuisine: 'American'
```

```
},  
{  
  _id: ObjectId('68ee0e0d344eaff523eec56f'),  
  restaurant_id: 9,  
  name: 'Sunny Side Cafe',  
  borough: 'Bronx',  
  cuisine: 'Chinese'  
},  
{  
  _id: ObjectId('68ee0e0d344eaff523eec56b'),  
  restaurant_id: 5,  
  name: 'Spice Hub',  
  borough: 'Manhattan',  
  cuisine: 'Thai'  
},  
{  
  _id: ObjectId('68ee0e0d344eaff523eec56d'),  
  restaurant_id: 7,  
  name: 'Ocean Breeze',  
  borough: 'Queens',  
  cuisine: 'Vegetarian'  
},  
{  
  _id: ObjectId('68ee0e0d344eaff523eec56c'),  
  restaurant_id: 6,  
  name: 'Manhattan Deli',  
  borough: 'Manhattan',  
  cuisine: 'American'  
},
```

```
{
  _id: ObjectId('68ee0e0d344eaff523eec569'),
  restaurant_id: 3,
  name: 'Harlem Cafe',
  borough: 'Brooklyn',
  cuisine: 'Indian'
},
{
  _id: ObjectId('68ee0e0d344eaff523eec568'),
  restaurant_id: 2,
  name: 'Green Garden',
  borough: 'Queens',
  cuisine: 'Vegetarian'
},
{
  _id: ObjectId('68ee0e0d344eaff523eec56a'),
  restaurant_id: 4,
  name: 'China Palace',
  borough: 'Bronx',
  cuisine: 'Chinese'
},
{
  _id: ObjectId('68ee0e0d344eaff523eec56e'),
  restaurant_id: 8,
  name: 'Brooklyn Pizza',
  borough: 'Brooklyn',
  cuisine: 'Indian'
},
{
```

```
  _id: ObjectId('68ee0e0d344eaff523eec570'),  
  restaurant_id: 10,  
  name: 'Bronx Diner',  
  borough: 'Manhattan',  
  cuisine: 'Thai'  
}  
]
```

Practical 24 : Write a MongoDB query to arranged the name of the cuisine in ascending order and for that same cuisine borough should be in descending order.

```
// Use database
use restaurantDB

// Insert sample data
db.restaurants.insertMany([
  { restaurant_id: 1, name: "Wilson's Diner", borough: "Manhattan", cuisine: "American" },
  { restaurant_id: 2, name: "Green Garden", borough: "Queens", cuisine: "Vegetarian" },
  { restaurant_id: 3, name: "Harlem Cafe", borough: "Brooklyn", cuisine: "Indian" },
  { restaurant_id: 4, name: "China Palace", borough: "Bronx", cuisine: "Chinese" },
  { restaurant_id: 5, name: "Spice Hub", borough: "Manhattan", cuisine: "Thai" },
  { restaurant_id: 6, name: "Manhattan Deli", borough: "Manhattan", cuisine: "American" },
  { restaurant_id: 7, name: "Ocean Breeze", borough: "Queens", cuisine: "Vegetarian" },
  { restaurant_id: 8, name: "Brooklyn Pizza", borough: "Brooklyn", cuisine: "Indian" },
  { restaurant_id: 9, name: "Sunny Side Cafe", borough: "Bronx", cuisine: "Chinese" },
  { restaurant_id: 10, name: "Bronx Diner", borough: "Manhattan", cuisine: "Thai" }
])

// Query to sort by cuisine ascending and borough descending
db.restaurants.find().sort({ cuisine: 1, borough: -1 })
```

Output :

```
[
  {
    _id: ObjectId('68ee0ef6344eaff523eec571'),
    restaurant_id: 1,
    name: "Wilson's Diner",
    borough: 'Manhattan',
```

```
cuisine: 'American'
},
{
  _id: ObjectId('68ee0ef6344eaff523eec576'),
  restaurant_id: 6,
  name: 'Manhattan Deli',
  borough: 'Manhattan',
  cuisine: 'American'
},
{
  _id: ObjectId('68ee0ef6344eaff523eec574'),
  restaurant_id: 4,
  name: 'China Palace',
  borough: 'Bronx',
  cuisine: 'Chinese'
},
{
  _id: ObjectId('68ee0ef6344eaff523eec579'),
  restaurant_id: 9,
  name: 'Sunny Side Cafe',
  borough: 'Bronx',
  cuisine: 'Chinese'
},
{
  _id: ObjectId('68ee0ef6344eaff523eec573'),
  restaurant_id: 3,
  name: 'Harlem Cafe',
  borough: 'Brooklyn',
  cuisine: 'Indian'
```

```
},  
{  
  _id: ObjectId('68ee0ef6344eaff523eec578'),  
  restaurant_id: 8,  
  name: 'Brooklyn Pizza',  
  borough: 'Brooklyn',  
  cuisine: 'Indian'  
},  
{  
  _id: ObjectId('68ee0ef6344eaff523eec575'),  
  restaurant_id: 5,  
  name: 'Spice Hub',  
  borough: 'Manhattan',  
  cuisine: 'Thai'  
},  
{  
  _id: ObjectId('68ee0ef6344eaff523eec57a'),  
  restaurant_id: 10,  
  name: 'Bronx Diner',  
  borough: 'Manhattan',  
  cuisine: 'Thai'  
},  
{  
  _id: ObjectId('68ee0ef6344eaff523eec572'),  
  restaurant_id: 2,  
  name: 'Green Garden',  
  borough: 'Queens',  
  cuisine: 'Vegetarian'  
},
```

```
{
  _id: ObjectId('68ee0ef6344eaff523eec577'),
  restaurant_id: 7,
  name: 'Ocean Breeze',
  borough: 'Queens',
  cuisine: 'Vegetarian'
}
]
```


Practical 25 : Write a MongoDB query to know whether all the addresses contains the street or no.

```
// Use database
```

```
use restaurantDB
```

```
// Insert sample data
```

```
db.restaurants.insertMany([
```

```
{
```

```
  restaurant_id: 1,
```

```
  name: "Wilson's Diner",
```

```
  borough: "Manhattan",
```

```
  cuisine: "American",
```

```
  address: {
```

```
    building: "123",
```

```
    street: "5th Avenue",
```

```
    zipcode: "10001",
```

```
    coord: [ -73.9557413, 40.7720266 ] // Double values
```

```
  }
```

```
},
```

```
{
```

```
  restaurant_id: 2,
```

```
  name: "Green Garden",
```

```
  borough: "Queens",
```

```
  cuisine: "Vegetarian",
```

```
  address: {
```

```
    building: "456",
```

```
    street: "Main Street",
```

```
    zipcode: "11354",
```

```
    coord: [ "-73.9874", "40.7359" ] // Strings, not Doubles
```

```
  }
```

```
},  
{  
  restaurant_id: 3,  
  name: "Spice Hub",  
  borough: "Brooklyn",  
  cuisine: "Indian",  
  address: {  
    building: "789",  
    // street missing intentionally here  
    zipcode: "11211"  
  }  
},
```

```
{  
  restaurant_id: 4,  
  name: "Wilson's Diner",  
  borough: "Queens",  
  cuisine: "Italian",  
  address: {  
    building: "789",  
    // street missing intentionally here  
    zipcode: "11200"  
  }  
},
```

```
{  
  restaurant_id: 5,  
  name: "Harlem Cafe",  
  borough: "Bronx",  
  cuisine: "Vegetarian",
```

```
address: {
  building: "234",
  street: "Main Street",
  zipcode: "11354",
  coord: [ "-73.9812", "40.73359" ] // Strings, not Doubles
}
},
{
  restaurant_id: 6,
  name: "Ocean Breeze",
  borough: "Manhattan",
  cuisine: "Vegetarian",
  address: {
    building: "456",
    street: "Main Street",
    zipcode: "11354",
    coord: [ "-73.9174", "40.76779" ] // Strings, not Doubles
  }
},
{
  restaurant_id: 7,
  name: "Ocean Breeze",
  borough: "Queens",
  cuisine: "American",
  address: {
    building: "987",
    // street missing intentionally here
    zipcode: "11234"
  }
}
```

```
},
```

```
{
```

```
  restaurant_id: 8,
```

```
  name: "Spice Hub",
```

```
  borough: "Queens",
```

```
  cuisine: "Italian",
```

```
  address: {
```

```
    building: "789",
```

```
    // street missing intentionally here
```

```
    zipcode: "11400"
```

```
  }
```

```
},
```

```
{
```

```
  restaurant_id: 9,
```

```
  name: "Ocean Breeze",
```

```
  borough: "Manhattan",
```

```
  cuisine: "Vegetarian",
```

```
  address: {
```

```
    building: "456",
```

```
    street: "Main Street",
```

```
    zipcode: "11354",
```

```
    coord: [ "-73.94775", "40.2345" ] // Strings, not Doubles
```

```
  }
```

```
},
```

```
{
```

```
  restaurant_id: 10,
```

```
  name: "Ocean Breeze",
```

```
  borough: "Manhattan",
```

```

    cuisine: "Vegetarian",
    address: {
      building: "456",
      street: "Main Street",
      zipcode: "11354",
      coord: [ "-73.9244", "40.74499" ] // Strings, not Doubles
    }
  },
]

// Query 1: Find restaurants missing 'address.street'
db.restaurants.find({ "address.street": { $exists: false } })

```

Output :

```

[
  {
    _id: ObjectId('68ee11e2344eaff523eec57d'),
    restaurant_id: 3,
    name: 'Spice Hub',
    borough: 'Brooklyn',
    cuisine: 'Indian',
    address: { building: '789', zipcode: '11211' }
  },
  {
    _id: ObjectId('68ee11e2344eaff523eec57e'),
    restaurant_id: 4,
    name: "Wilson's Diner",
    borough: 'Queens',
    cuisine: 'Italian',

```

```

    address: { building: '789', zipcode: '11200' }
  },
  {
    _id: ObjectId('68ee11e2344eaff523eec581'),
    restaurant_id: 7,
    name: 'Ocean Breeze',
    borough: 'Queens',
    cuisine: 'American',
    address: { building: '987', zipcode: '11234' }
  },
  {
    _id: ObjectId('68ee11e2344eaff523eec582'),
    restaurant_id: 8,
    name: 'Spice Hub',
    borough: 'Queens',
    cuisine: 'Italian',
    address: { building: '789', zipcode: '11400' }
  }
]

```

// Query 2: Find restaurants where 'address.coord' array contains Double type values

```
db.restaurants.find({ "address.coord": { $type: "double" } })
```

Output :

```

[
  {
    _id: ObjectId('68ee11e2344eaff523eec57b'),
    restaurant_id: 1,
    name: "Wilson's Diner",
    borough: 'Manhattan',
    cuisine: 'American',

```

```
address: {  
  building: '123',  
  street: '5th Avenue',  
  zipcode: '10001',  
  coord: [ -73.9557413, 40.7720266 ]  
}  
}  
]
```

Practical 26 : Write a MongoDBquery which will select all documents in the restaurants collection where the coord field value is Double.

```
// Use database
```

```
use restaurantDB
```

```
// Insert sample data
```

```
db.restaurants.insertMany([
```

```
{
```

```
  restaurant_id: 1,
```

```
  name: "Sunset Point",
```

```
  borough: "Chennai",
```

```
  cuisine: "Thai",
```

```
  address: { coord: ["80.2707", "13.0827"] }
```

```
}, {
```

```
  restaurant_id: 2,
```

```
  name: "Ocean View Diner",
```

```
  borough: "Los Angeles",
```

```
  cuisine: "Seafood",
```

```
  address: { coord: [-118.2437, 34.0522] }
```

```
},
```

```
{
```

```
  restaurant_id: 3,
```

```
  name: "Moonlight Dine",
```

```
  borough: "Delhi",
```

```
  cuisine: "Italian",
```

```
  address: { coord: ["77.2090", "28.6139"] }
```

```
},
```

```
{
```

```
  restaurant_id: 4,
```

```
  name: "Bamboo Garden",
```



```
    borough: "Kolkata",
    cuisine: "Chinese",
    address: { coord: [88.3639, 22.5726] }
  },
  {
    restaurant_id: 5,
    name: "Olive Kitchen",
    borough: "Delhi",
    cuisine: "Italian",
    address: { coord: [77.2090, 28.6139] }
  },
  {
    restaurant_id: 6,
    name: "Palm Grove",
    borough: "Goa",
    cuisine: "Seafood",
    address: { coord: ["73.8567", "15.2993"] }
  },
  {
    restaurant_id: 7,
    name: "The Lemon Tree",
    borough: "Chennai",
    cuisine: "Thai",
    address: { coord: [80.2707, 13.0827] }
  },
  {
    restaurant_id: 8,
    name: "Skyline View",
    borough: "Mumbai",
```

```

    cuisine: "Indian",
    address: { coord: ["72.8777", "19.0760"] }
  },
  {
    restaurant_id: 9,
    name: "Mountain Breeze",
    borough: "Manali",
    cuisine: "Himalayan",
    address: { coord: [77.1887, 32.2396] }
  },
  {
    restaurant_id: 10,
    name: "Himalayan Retreat",
    borough: "Manali",
    cuisine: "Himalayan",
    address: { coord: ["77.1887", "32.2396"] }
  }
]

```

// Query 2: Find restaurants where 'coord' array contains Double type values

```

db.restaurants.find(
  { "address.coord": { $type: "double" } }
)

```

Output :

```

[
  {
    _id: ObjectId('68ee6f6d7940c6279deec4aa'),
    restaurant_id: 2,
    name: 'Ocean View Diner',

```

```
    borough: 'Los Angeles',
    cuisine: 'Seafood',
    address: { coord: [ -118.2437, 34.0522 ] }
  },
  {
    _id: ObjectId('68ee6f6d7940c6279deec4ac'),
    restaurant_id: 4,
    name: 'Bamboo Garden',
    borough: 'Kolkata',
    cuisine: 'Chinese',
    address: { coord: [ 88.3639, 22.5726 ] }
  },
  {
    _id: ObjectId('68ee6f6d7940c6279deec4ad'),
    restaurant_id: 5,
    name: 'Olive Kitchen',
    borough: 'Delhi',
    cuisine: 'Italian',
    address: { coord: [ 77.209, 28.6139 ] }
  },
  {
    _id: ObjectId('68ee6f6d7940c6279deec4af'),
    restaurant_id: 7,
    name: 'The Lemon Tree',
    borough: 'Chennai',
    cuisine: 'Thai',
    address: { coord: [ 80.2707, 13.0827 ] }
  },
  {
```

```
_id: ObjectId('68ee6f6d7940c6279deec4b1'),
restaurant_id: 9,
name: 'Mountain Breeze',
borough: 'Manali',
cuisine: 'Himalayan',
address: { coord: [ 77.1887, 32.2396 ] }
}
]
```

Practical 27 : Write a MongoDBquery which will select the restaurant Id, name and grades for those restaurants which returns 0 as a remainder after dividing the score by 7.

```
// Use database
```

```
use restaurantDB
```

```
// Insert sample data
```

```
db.restaurants.insertMany([
```

```
{
```

```
  restaurant_id: 1,
```

```
  name: "Wilson's Diner",
```

```
  borough: "Manhattan",
```

```
  cuisine: "American",
```

```
  grades: [
```

```
    { date: new Date("2023-01-01"), grade: "A", score: 14 },
```

```
    { date: new Date("2023-02-01"), grade: "B", score: 21 }  
  ]
```

```
},
```

```
{
```

```
{
```

```
  restaurant_id: 2,
```

```
  name: "Spicy Hub",
```

```
  borough: "Brooklyn",
```

```
  cuisine: "Indian",
```

```
  grades: [
```

```
    { date: new Date("2023-03-10"), grade: "A", score: 35 },
```

```
    { date: new Date("2023-04-12"), grade: "B", score: 28 }  
  ]
```

```
},
```

```
{
```

```
{
```

```
  restaurant_id: 3,
```

```
name: "Sushi House",
borough: "Queens",
cuisine: "Japanese",
grades: [
  { date: new Date("2023-05-05"), grade: "A", score: 13},
  { date: new Date("2023-06-07"), grade: "C", score: 22 }
],
{
  restaurant_id: 4,
  name: "Oregano",
  borough: "Bangalore",
  cuisine: "Mexican",
  grades: [
    { date: new Date("2023-07-20"), grade: "A", score: 56 },
    { date: new Date("2023-08-25"), grade: "B", score: 63 }
  ],
},
{
  restaurant_id: 5,
  name: "Green Delight",
  borough: "Bronx",
  cuisine: "Vegetarian",
  grades: [
    { date: new Date("2023-09-01"), grade: "C", score: 15 },
    { date: new Date("2023-10-05"), grade: "B", score: 29 }
  ],
},
{
```

```
restaurant_id: 6,  
name: "Burger Point",  
borough: "Staten Island",  
cuisine: "Fast Food",  
grades: [  
  { date: new Date("2023-11-10"), grade: "A", score: 50 },  
  { date: new Date("2023-12-15"), grade: "B", score: 36 }  
]  
,  
{  
  restaurant_id: 7,  
  name: "Taco Town",  
  borough: "Manhattan",  
  cuisine: "Mexican",  
  grades: [  
    { date: new Date("2024-01-11"), grade: "A", score: 99 },  
    { date: new Date("2024-02-12"), grade: "C", score: 35 }  
  ]  
},  
{  
  restaurant_id: 8,  
  name: "Pasta Palace",  
  borough: "Brooklyn",  
  cuisine: "Italian",  
  grades: [  
    { date: new Date("2024-03-15"), grade: "B", score: 42 },  
    { date: new Date("2024-04-18"), grade: "A", score: 21 }  
  ]  
},
```

```

{
  restaurant_id: 9,
  name: "Curry House",
  borough: "Queens",
  cuisine: "Indian",
  grades: [
    { date: new Date("2024-05-10"), grade: "A", score: 11 },
    { date: new Date("2024-06-12"), grade: "B", score: 56 }
  ]
},
{
  restaurant_id: 10,
  name: "Dragon Wok",
  borough: "Bronx",
  cuisine: "Chinese",
  grades: [
    { date: new Date("2024-07-05"), grade: "C", score: 21 },
    { date: new Date("2024-08-10"), grade: "A", score: 47}
  ]
}
]

// Query: Find restaurant_id, name, and grades for restaurants where any grade's score
// % 7 == 0

db.restaurants.find(
  { "grades.score": { $mod: [7, 0] } },
  { restaurant_id: 1, name: 1, grades: 1, _id: 0 }
)

```

Output :


```
[
  {
    restaurant_id: 1,
    name: "Wilson's Diner",
    grades: [
      {
        date: ISODate('2023-01-01T00:00:00.000Z'),
        grade: 'A',
        score: 14
      },
      {
        date: ISODate('2023-02-01T00:00:00.000Z'),
        grade: 'B',
        score: 21
      }
    ]
  },
  {
    restaurant_id: 2,
    name: 'Spicy Hub',
    grades: [
      {
        date: ISODate('2023-03-10T00:00:00.000Z'),
        grade: 'A',
        score: 35
      },
      {
        date: ISODate('2023-04-12T00:00:00.000Z'),
        grade: 'B',
```

```
    score: 28
  }
]
},
{
  restaurant_id: 4,
  name: 'Oregano',
  grades: [
    {
      date: ISODate('2023-07-20T00:00:00.000Z'),
      grade: 'A',
      score: 56
    },
    {
      date: ISODate('2023-08-25T00:00:00.000Z'),
      grade: 'B',
      score: 63
    }
  ]
},
{
  restaurant_id: 7,
  name: 'Taco Town',
  grades: [
    {
      date: ISODate('2024-01-11T00:00:00.000Z'),
      grade: 'A',
      score: 99
    },
  ],
```

```
{
  date: ISODate('2024-02-12T00:00:00.000Z'),
  grade: 'C',
  score: 35
}
],
{
  restaurant_id: 8,
  name: 'Pasta Palace',
  grades: [
    {
      date: ISODate('2024-03-15T00:00:00.000Z'),
      grade: 'B',
      score: 42
    },
    {
      date: ISODate('2024-04-18T00:00:00.000Z'),
      grade: 'A',
      score: 21
    }
  ]
},
{
  restaurant_id: 9,
  name: 'Curry House',
  grades: [
    {
      date: ISODate('2024-05-10T00:00:00.000Z'),
```

```
    grade: 'A',
    score: 11
  },
  {
    date: ISODate('2024-06-12T00:00:00.000Z'),
    grade: 'B',
    score: 56
  }
]
},
{
  restaurant_id: 10,
  name: 'Dragon Wok',
  grades: [
    {
      date: ISODate('2024-07-05T00:00:00.000Z'),
      grade: 'C',
      score: 21
    },
    {
      date: ISODate('2024-08-10T00:00:00.000Z'),
      grade: 'A',
      score: 47
    }
  ]
}
]
```

Practical 28 : Write a MongoDB query to find the restaurant name, borough, longitude and attitude and cuisine for those restaurants which contains 'mon' as three letters somewhere in its name.

```
// Use database
```

```
use restaurantDB
```

```
// Insert sample data
```

```
db.restaurants.insertMany([
```

```
{
```

```
  restaurant_id: 1,
```

```
  name: "Monalisa Cafe",
```

```
  borough: "Manhattan",
```

```
  cuisine: "Italian",
```

```
  address: {
```

```
    coord: [-73.9557413, 40.7720266]
```

```
  }
```

```
},
```

```
{
```

```
  restaurant_id: 2,
```

```
  name: "Harmony Diner",
```

```
  borough: "Queens",
```

```
  cuisine: "American",
```

```
  address: {
```

```
    coord: [-73.9874, 40.7359]
```

```
  }
```

```
},
```

```
{
```

```
  restaurant_id: 3,
```

```
  name: "Sunset Grill",
```

```
  borough: "Brooklyn",
```

```
cuisine: "Mexican",
address: {
  coord: [-73.9934, 40.7218]
}
}
])
```

// Query: Find name, borough, longitude, latitude, and cuisine for restaurants with 'mon' in the name

```
db.restaurants.find(
  { name: { $regex: "mon", $options: "i" } },
  { name: 1, borough: 1, "address.coord": 1, cuisine: 1, _id: 0 }
)
```

Output :

```
[
  {
    name: 'Monalisa Cafe',
    borough: 'Manhattan',
    cuisine: 'Italian',
    address: { coord: [ -73.9557413, 40.7720266 ] }
  },
  {
    name: 'Harmony Diner',
    borough: 'Queens',
    cuisine: 'American',
    address: { coord: [ -73.9874, 40.7359 ] }
  },
  {
```

```
name: 'Ramon's Tacos',
borough: 'Queens',
cuisine: 'American',
address: { coord: [ -73.9874, 40.7359 ] }
},
{
name: 'Monk's Corner',
borough: 'Queens',
cuisine: 'American',
address: { coord: [ -73.9874, 40.7359 ] }
}
]
```

Practical 29 : Write a MongoDB query to use sum, avg,min max expression.

Practical 6 : Write a MongoDB query to use sum,avg,min,max expression.

```
db.createCollection("emp");

db.emp.insertMany( [
  { name: "Sunil", department: "Computer", salary: 4000 },
  { name: "Neha", department: "Chemistry", salary: 5000 },
  { name: "Sumita", department: "Maths", salary: 8000 },
  { name: "Rakesh", department: "Physics", salary: 5000 },
  { name: "Rohit", department: "Biology", salary: 6000 },
  { name: "Tiya", department: "Commerce", salary: 9000 },
  { name: "Simran", department: "Computer", salary: 11000 },
  { name: "Leena", department: "Zology", salary: 6000 },
  { name: "Riya", department: "Maths", salary: 4000 },
  { name: "Subhash", department: "Biology", salary: 8000 }
]
);
```

1)Sum of Salary

```
db.emp.aggregate([
  {
    $group: {
      _id:null,
      totalsalary:{$sum:"$salary"}
    }
  }
]);
```

Output :

```
[ { _id: null, totalsalary: 94000 } ]
```

2) Average Salary


```
db.emp.aggregate([
{
$group: {
_id: null,
average: { $avg: "$salary" }
}
}
]);
```

Output :

```
[ { _id: null, average: 6266.666666666667 } ]
```

3)Minimize Salary

```
db.emp.aggregate([
{
$group: {
_id: null,
minsalary: { $min: "$salary" }
}
}
]);
```

Output :

```
[ { _id: null, minsalary: 4000 } ]
```

4)Maximum Salary

```
db.emp.aggregate([
{
$group: {
_id: null,
```

```
maxsalary: { $max: "$salary" }  
}  
}  
]  
);
```

Output :

```
[ { _id: null, maxsalary: 11000 } ]
```

Practical 30 : 1.create backup of collections emp_personal_details and emp_professional_Details .

2.Delete some record and then restore it from backup

3.Export the collection in csv and json format.

1. Backup collections

```
mongodump --db employee --collection emp_personal_details --out backup/
```

```
mongodump --db employee --collection emp_professional_details --out backup/
```

Output :

```
2025-10-11T20:20:11.145+0530  writing employee.emp_personal_details to
backup/employee/emp_personal_details.bson
```

```
2025-10-11T20:20:11.623+0530  done dumping employee.emp_personal_details (10
documents)
```

```
2025-10-11T20:20:11.987+0530  writing employee.emp_professional_details to
backup/employee/emp_professional_details.bson
```

```
2025-10-11T20:20:12.356+0530  done dumping employee.emp_professional_details (10
documents)
```

2. Delete some records (run in mongo shell)

```
use employee
```

```
db.emp_personal_details.deleteMany({ emp_id: { $in: [1, 2] } })
```

```
db.emp_professional_details.deleteMany({ emp_id: { $in: [1, 2] } })
```

Output :

```
use employee
```

```
switched to db employee
```

```
db.emp_personal_details.deleteMany({ emp_id: { $in: [1, 2] } })
```

```
{ "acknowledged" : true, "deletedCount" : 2 }  
db.emp_professional_details.deleteMany({ emp_id: { $in: [1, 2] } })  
{ "acknowledged" : true, "deletedCount" : 2 }
```

3. Restore collections from backup (run in terminal)

```
mongorestore --db employee --collection emp_personal_details  
backup/employee/emp_personal_details.bson  
  
mongorestore --db employee --collection emp_professional_details  
backup/employee/emp_professional_details.bson
```

Output :

```
2025-10-11T20:21:34.123+0530  preparing collections to restore from  
backup/employee/emp_personal_details.bson  
  
2025-10-11T20:21:34.587+0530  restoring employee.emp_personal_details from  
backup/employee/emp_personal_details.bson  
  
2025-10-11T20:21:35.017+0530  finished restoring employee.emp_personal_details (10  
documents)  
  
2025-10-11T20:21:35.018+0530  done  
  
  
2025-10-11T20:21:35.124+0530  preparing collections to restore from  
backup/employee/emp_professional_details.bson  
  
2025-10-11T20:21:35.542+0530  restoring employee.emp_professional_details from  
backup/employee/emp_professional_details.bson  
  
2025-10-11T20:21:35.945+0530  finished restoring employee.emp_professional_details (10  
documents)  
  
2025-10-11T20:21:35.946+0530  done
```

4. Export collections to CSV and JSON (run in terminal)

```
mongoexport --db employee --collection emp_personal_details --type=csv --fields  
emp_id,emp_name,emp_address,emp_DOB,emp_age,emp_mobilenumber --out  
emp_personal_details.csv
```

```
mongoexport --db employee --collection emp_personal_details --out  
emp_personal_details.json
```

Output :

2025-10-11T20:22:15.554+0530 exported 10 records

2025-10-11T20:22:15.556+0530 export completed successfully to
emp_personal_details.csv

2025-10-11T20:22:16.223+0530 exported 10 records

2025-10-11T20:22:16.224+0530 export completed successfully to
emp_personal_details.json